

SAIR

Spatial AI & Robotics Lab

CSE 473/573

L14: TEXTURE & SEGMENTATION

Chen Wang

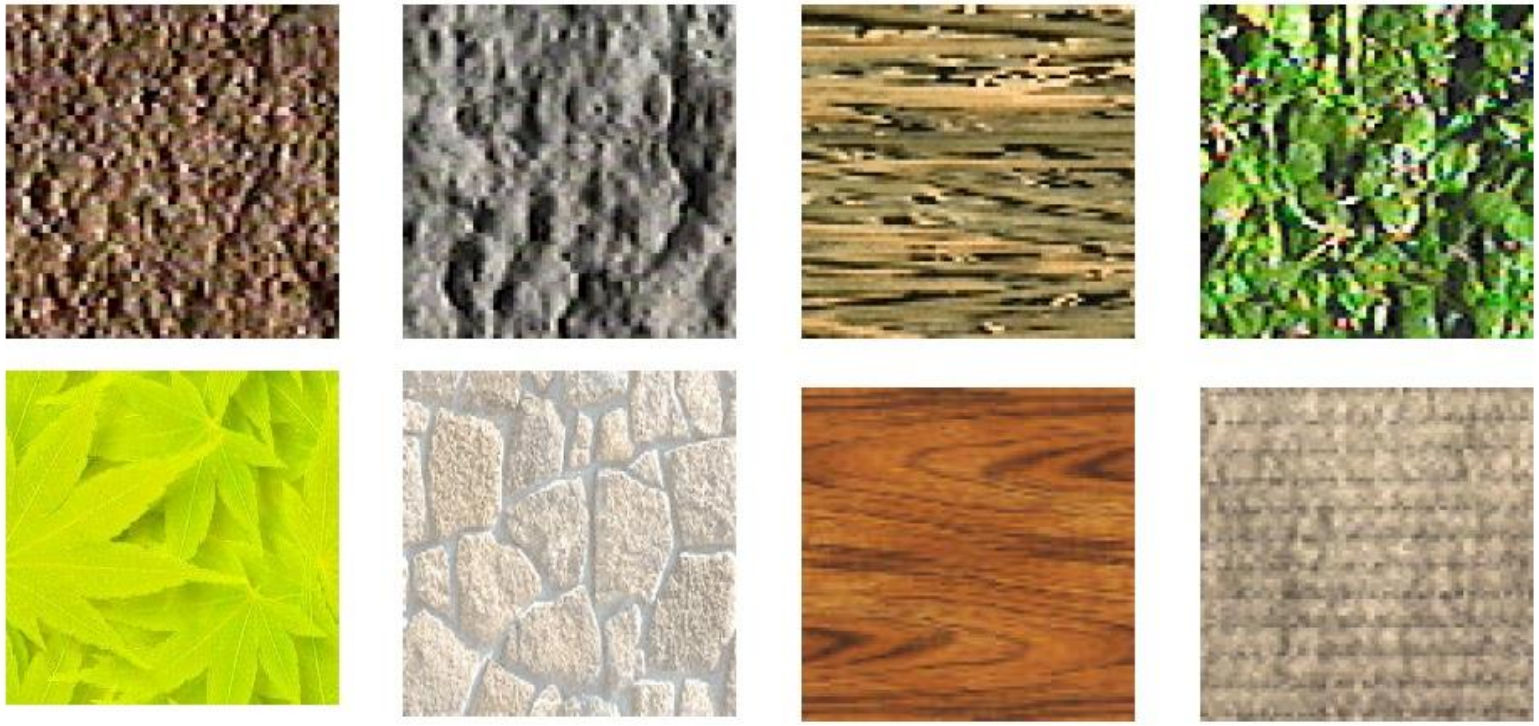
Spatial AI & Robotics Lab

Department of Computer Science and Engineering

University at Buffalo The State University of New York

Texture

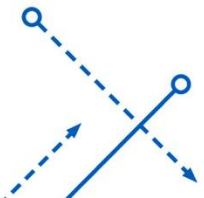
- What defines a texture?



What is Texture?

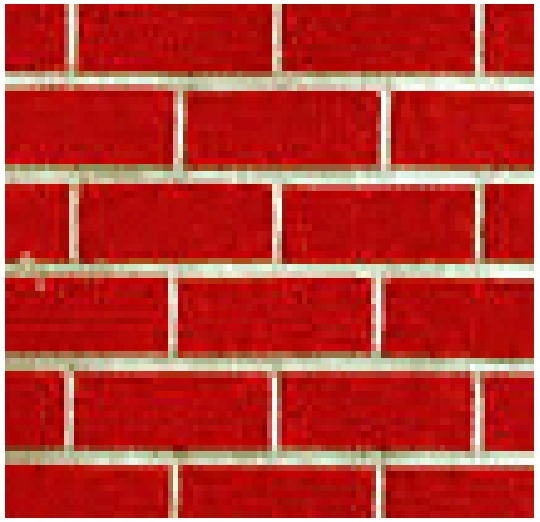
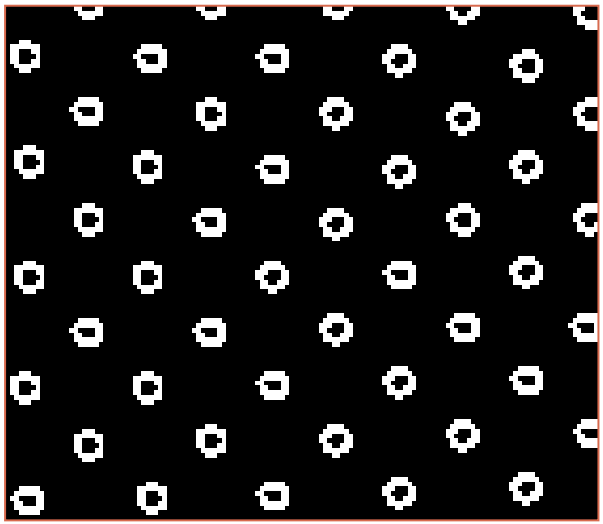
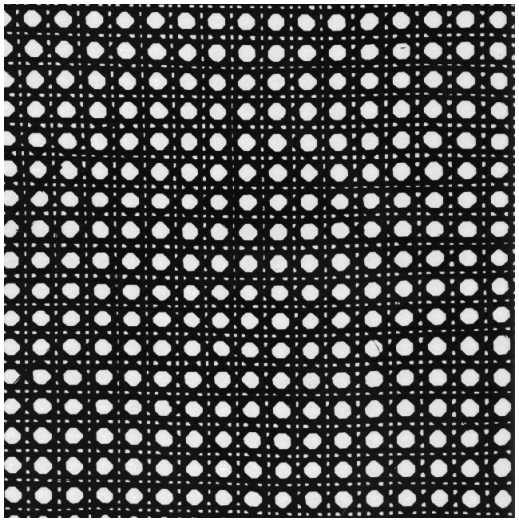
- No Formal Definition
  - There will be **signficant variation** in intensity levels between pixels
  - these variations perform **repetitive** patterns-homogeneous at some scale
  - **local statistics** are constant and slowly varying

- Human visual systems perceived textures as **homogeneous regions** even though they don't have the same intensity



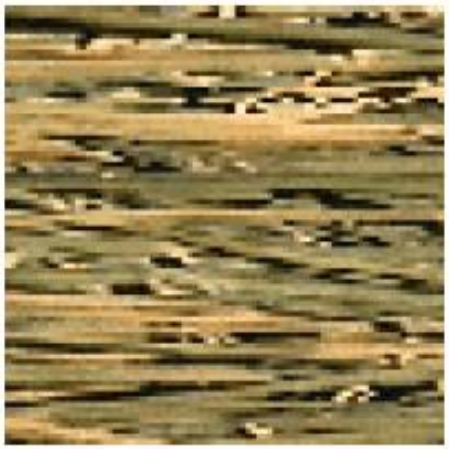
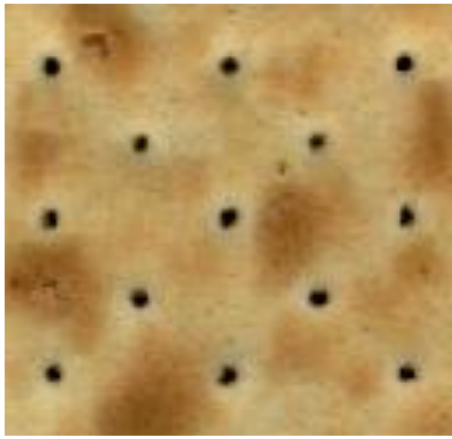
Regular (Repetitive) patterns

- We interpret this first image as a brick wall

Random patterns

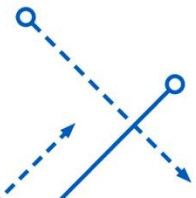


Texture

- Is a property of a "group of pixels" or Area
  - a single pixel does not have texture
- It is **scale dependent**
  - at different scales textures have different properties
- It contains many possibly countless **primitive objects**
- It involves the **spatial distribution** of intensities
  - 2D histograms
  - Co-occurrence matrices





Scale: objects vs. texture

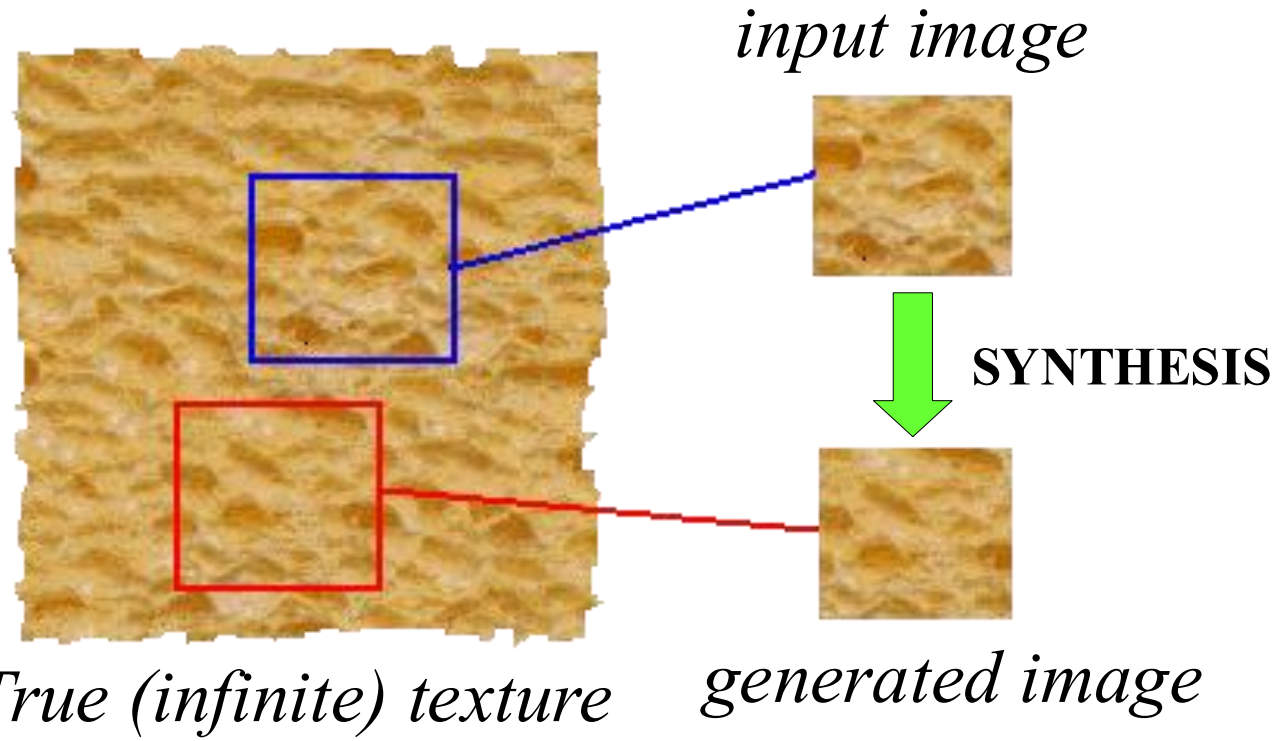
- The same thing can occur as texture or an object
  - depending on the **scale** we are considering.





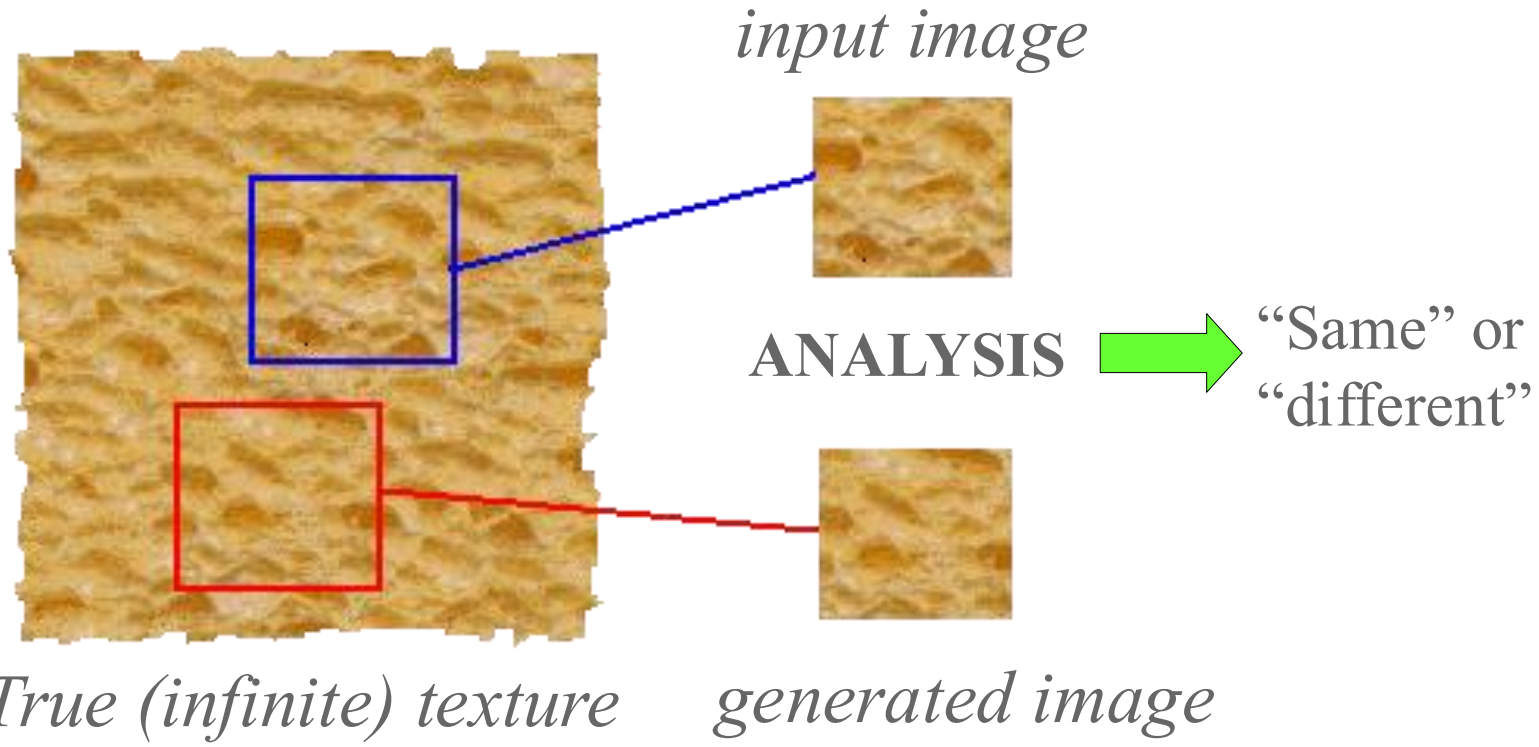
The Goal of Texture **Synthesis**

- Given a finite sample of some texture, the goal is to synthesize other samples from that same texture
- The sample needs to be "large enough“



The Goal of Texture **Analysis**

- Compare textures and decide if they're made of the same "stuff".



Texture-related tasks









Texture-related tasks





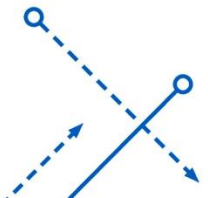




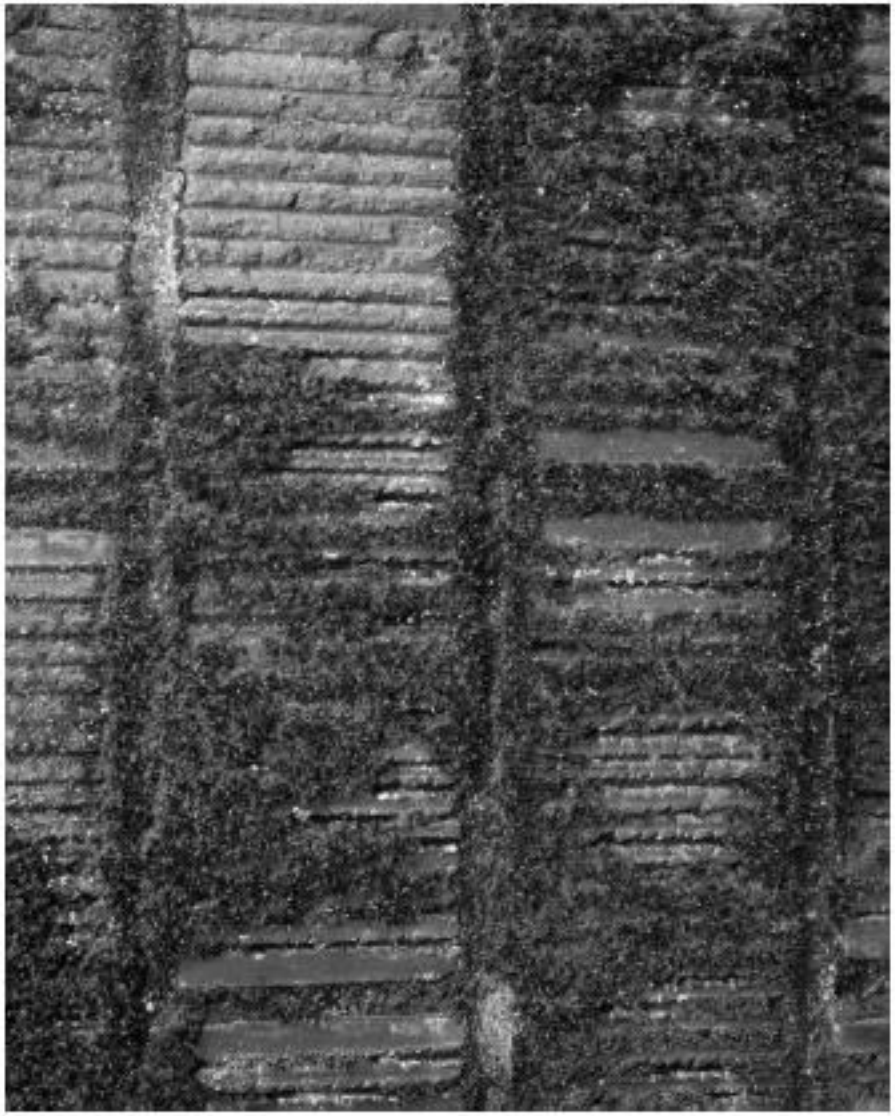
http://animals.nationalgeographic.com/

Texture representation

- Textures are made up of repeated local patterns:
  - How to find the patterns
    - Use filters that look like patterns
      - Spots, bars, raw patches…
      - Consider magnitude of response
    - Describe their statistics within each local window
      - Mean, standard deviation, etc.
    - Histogram of “prototypical” feature occurrences



Texture representation

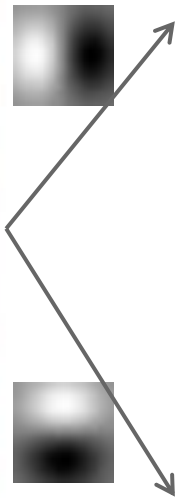


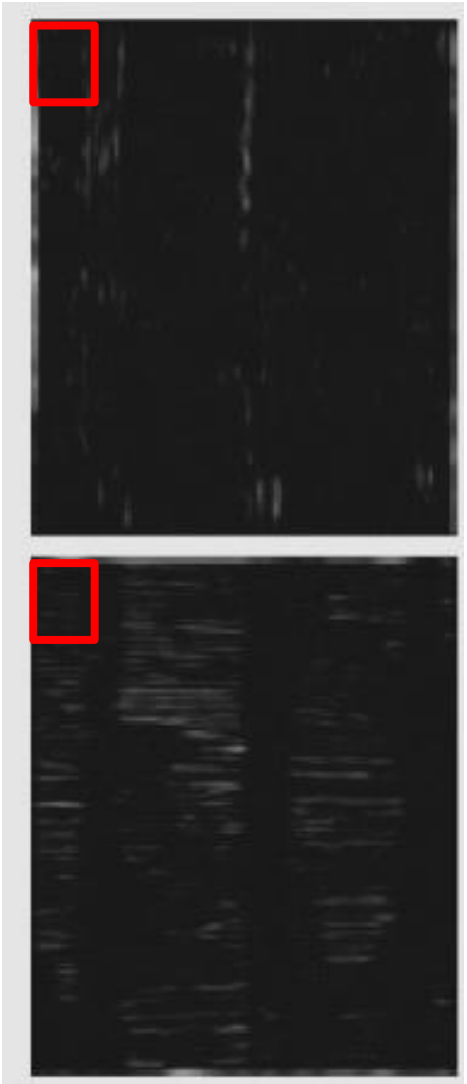


Texture representation



original image





derivative filter responses, squared

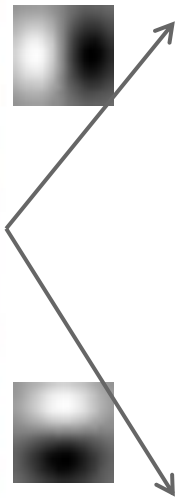
	mean d/dx value	mean d/dy value
Win. #1	4	10

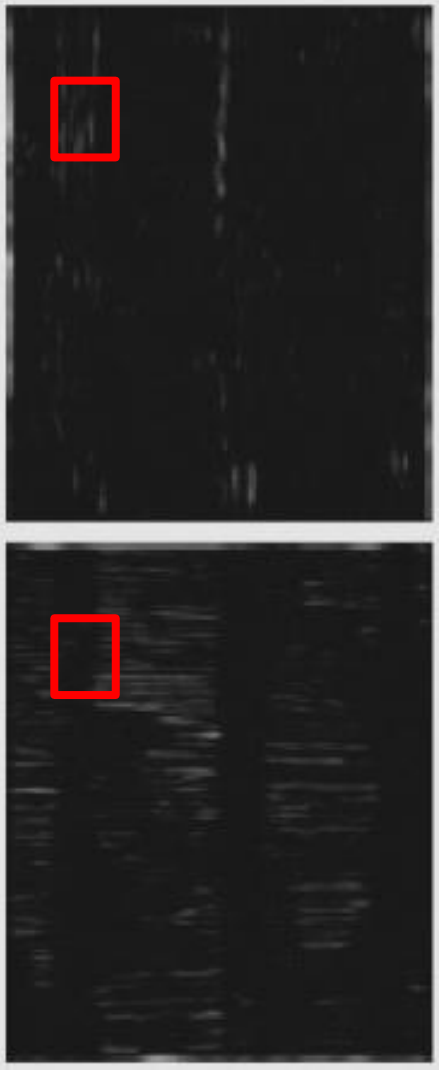
statistics to summarize patterns in small windows

Texture representation



original image





derivative filter responses, squared

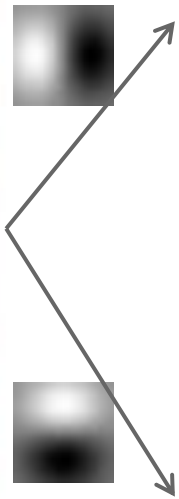
	mean d/dx value	mean d/dy value
Win. #1	4	10
Win.#2	18	7

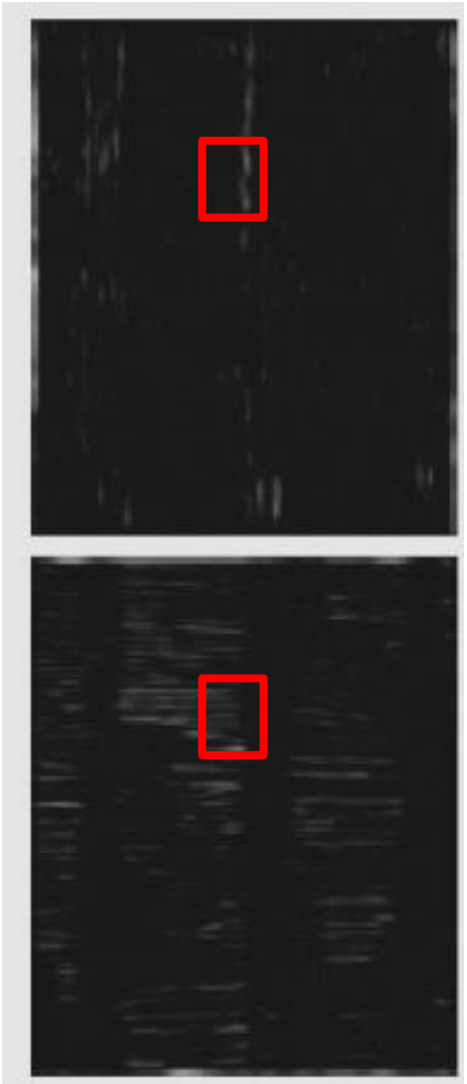
statistics to summarize patterns in small windows

Texture representation



original image





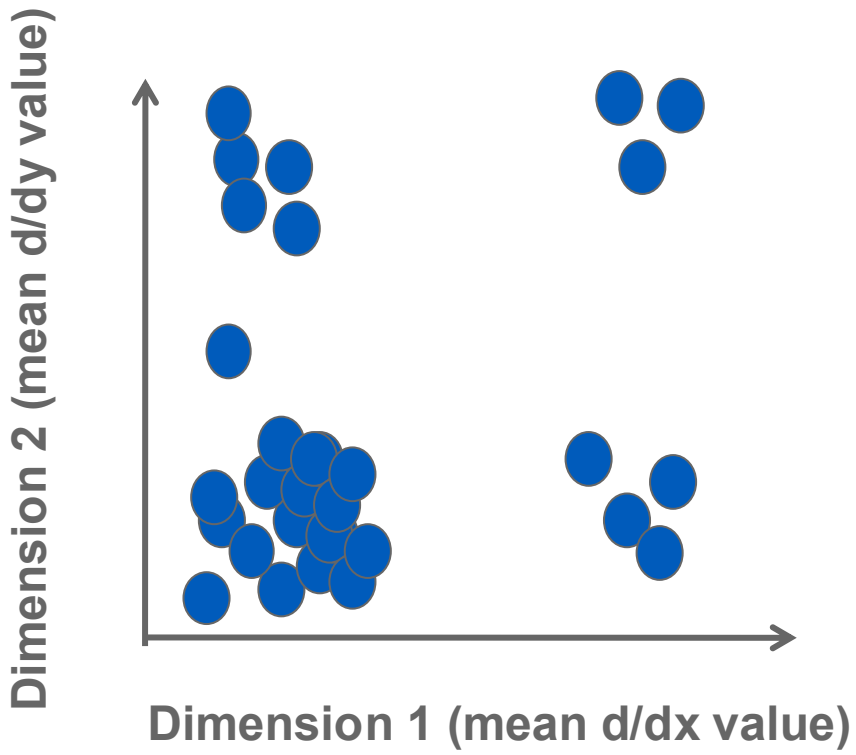
derivative filter responses, squared

	mean d/dx value	mean d/dy value
Win. #1	4	10
Win.#2	18	7
⋮		
Win.#9	20	20

⋮

statistics to summarize patterns in small windows

Texture representation: example

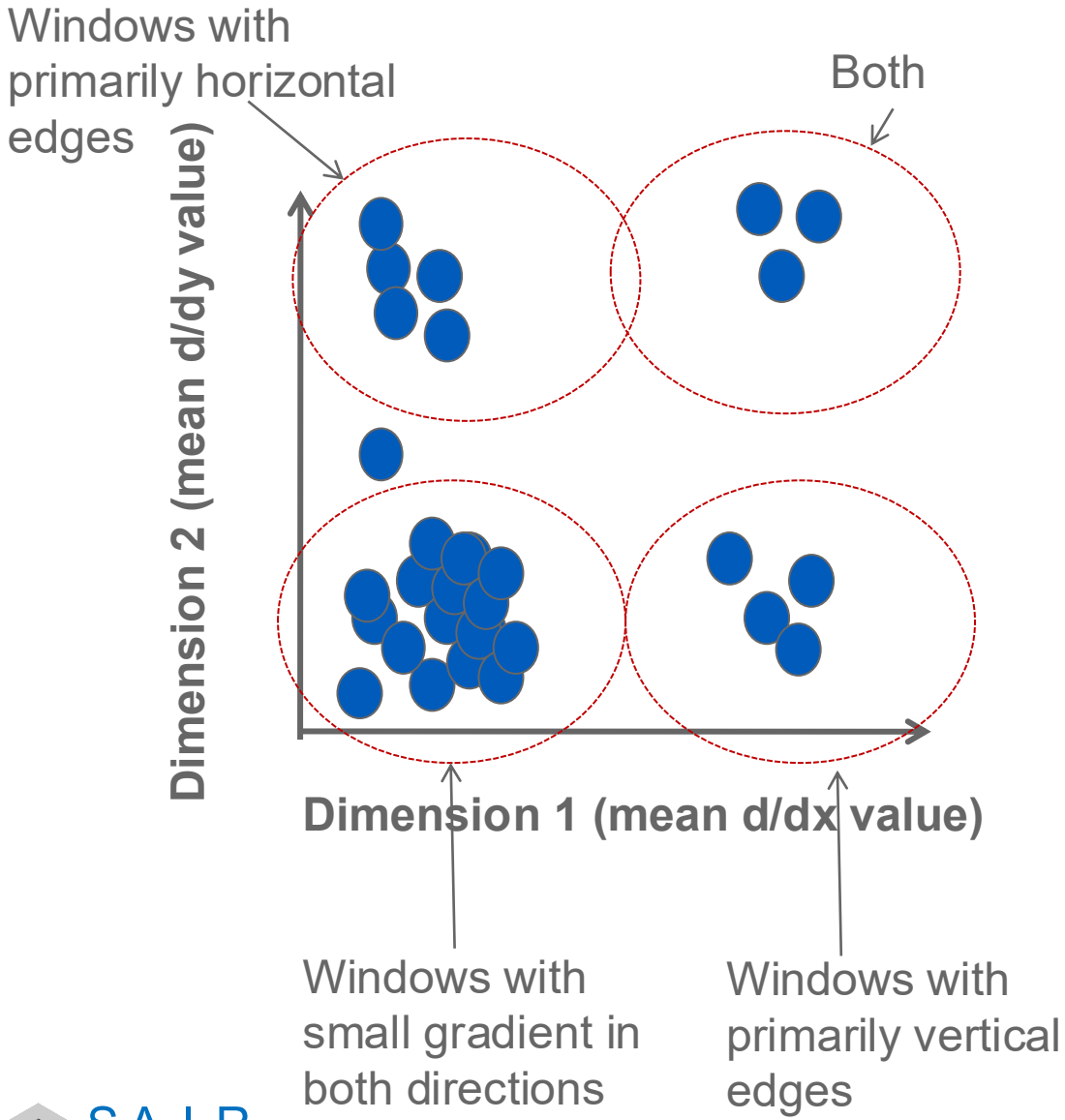


	mean d/dx value	mean d/dy value
Win. #1	4	10
Win.#2	18	7
⋮		
Win.#9	20	20

⋮

statistics to summarize patterns in small windows

Texture representation: example



	mean d/dx value	mean d/dy value
Win. #1	4	10
Win.#2	18	7
⋮		
Win.#9	20	20

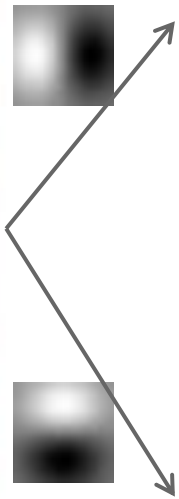
⋮

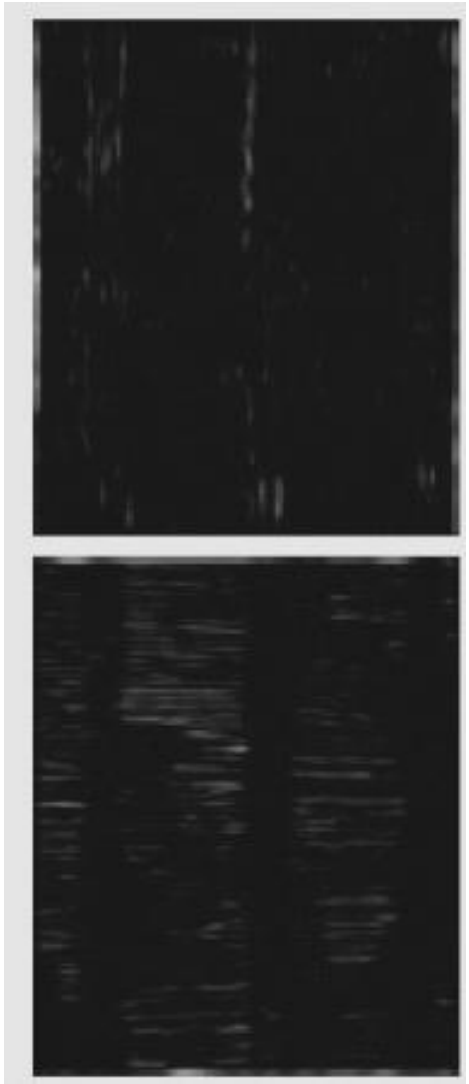
statistics to summarize patterns in small windows

Texture representation: example

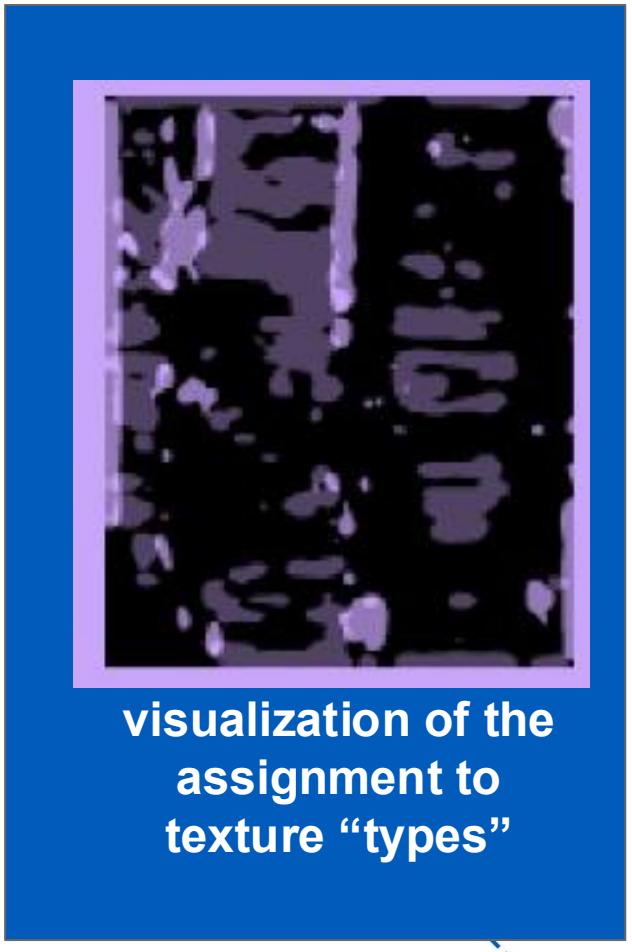


original image



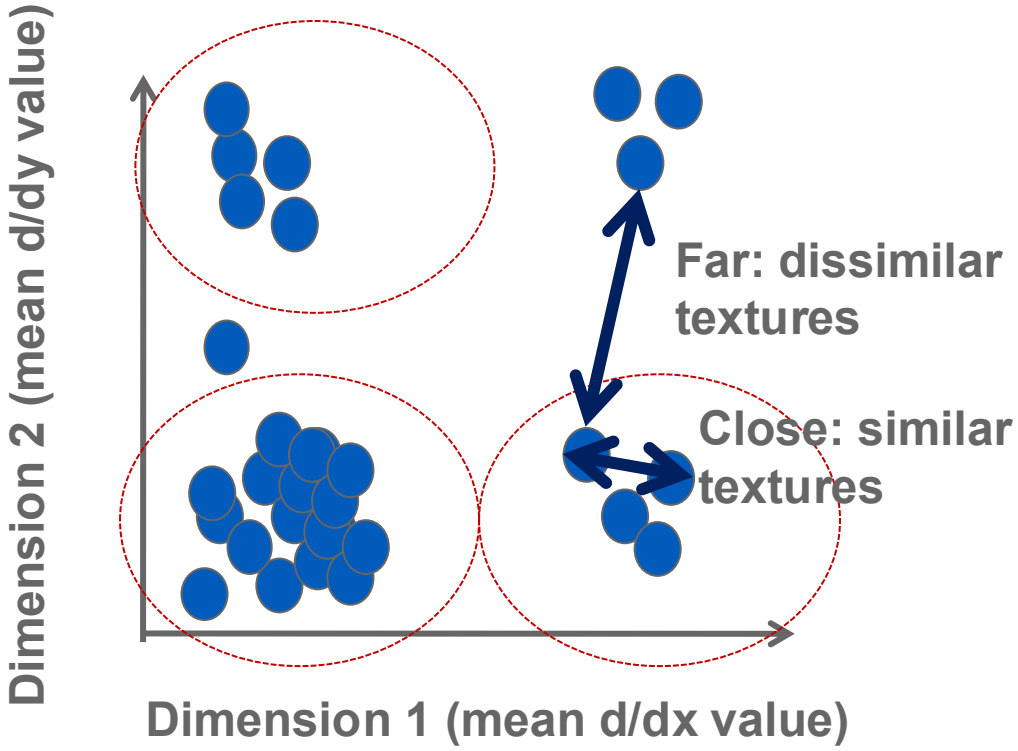


derivative filter responses, squared



visualization of the assignment to texture “types”

Texture representation: example



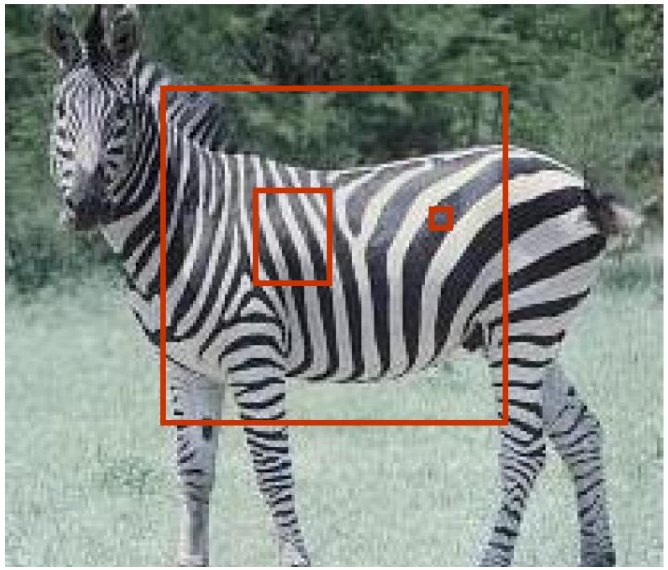
	mean d/dx value	mean d/dy value
Win. #1	4	10
Win.#2	18	7
⋮		
Win.#9	20	20

⋮

statistics to summarize patterns in small windows

Texture representation: window scale

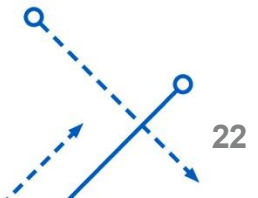
- We're assuming we know the relevant window size for which we collect these statistics.



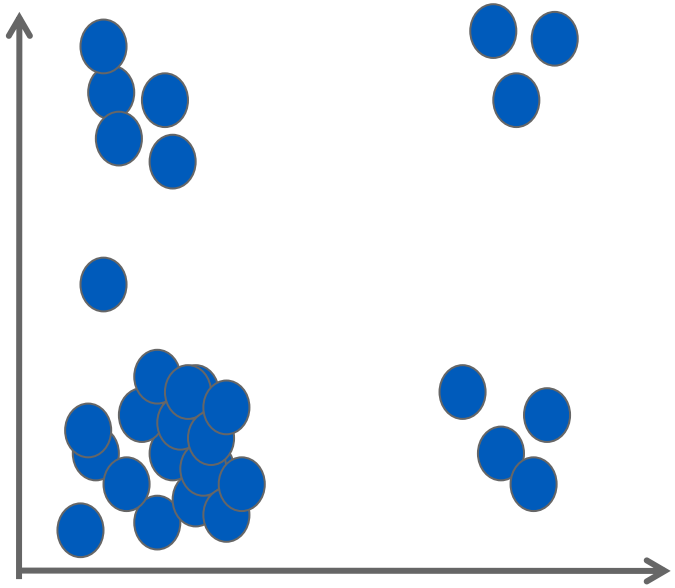
Possible to perform scale selection by looking for window scale where texture description not changing.

Filter banks

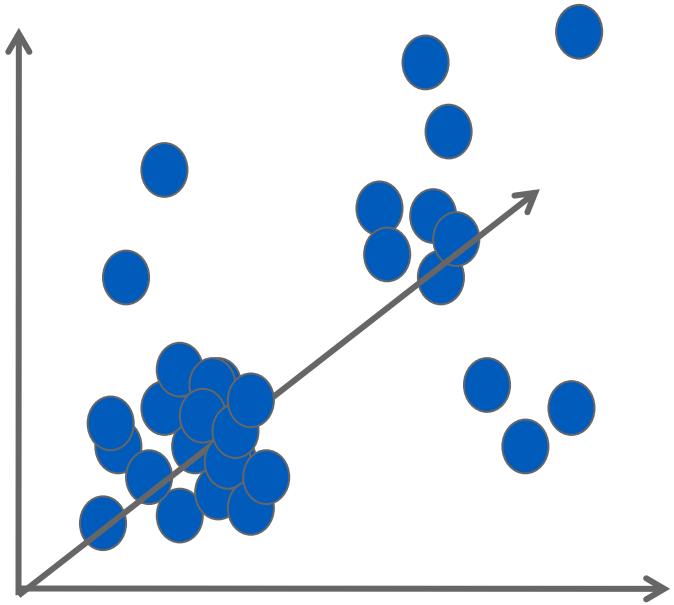
- Our previous example used two filters, and resulted in a 2-dimensional feature vector to describe texture in a window.
  - x and y derivatives revealed something about local structure.
- We can generalize to apply a collection of multiple (*d*) filters: a “filter bank”
  - Recall multi-channel convolution.
- Then our feature vectors will be *d*-dimensional.
  - still can think of nearness, farness in feature space



d-dimensional features



2d

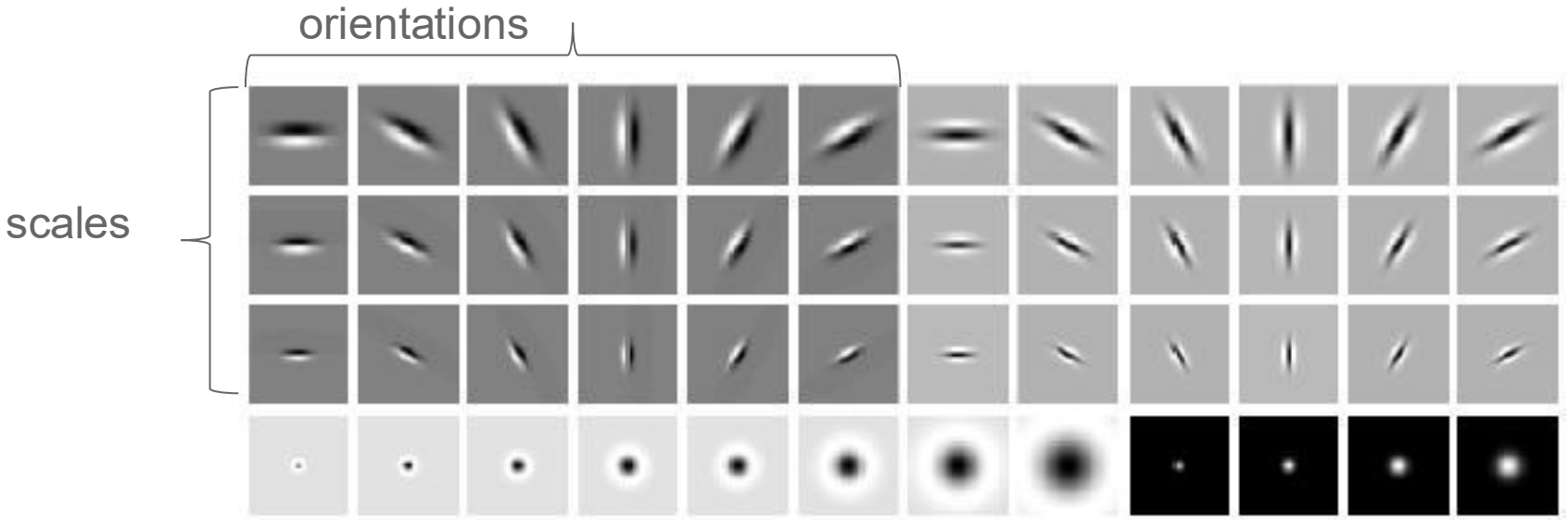


3d

. . .

Filter banks

- What filters to put in the bank?
  - Typically we want a combination of scales and orientations, different types of patterns.



http://www.robots.ox.ac.uk/~vgg/research/texclass/filters.html

Pixel Neighborhood-based Feature

- The most important for texture analysis is to describe the spatial behavior of intensity values in any neighborhood.
- Different methodologies have been proposed.
- **Local binary pattern (LBP)** is one of the most-widely used approach – mainly for face recognition.
- LBP is used for texture analysis too.

Local Binary Pattern (LBP)

For each PIXEL of an image, a BINARY CODE is produced

$$\mathrm{LBP}_{p,r}(N_c) = \sum_{p=0}^{P-1} g(N_p - N_c)\ 2^p$$

where

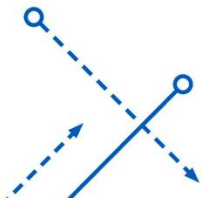
neighborhood pixels (N_p) in each block → is thresholded by its center pixel (N_c)

p→ sampling points (e.g., $p = 0, 1, \ldots, 7$ for a 3x3 cell, where P = 8)

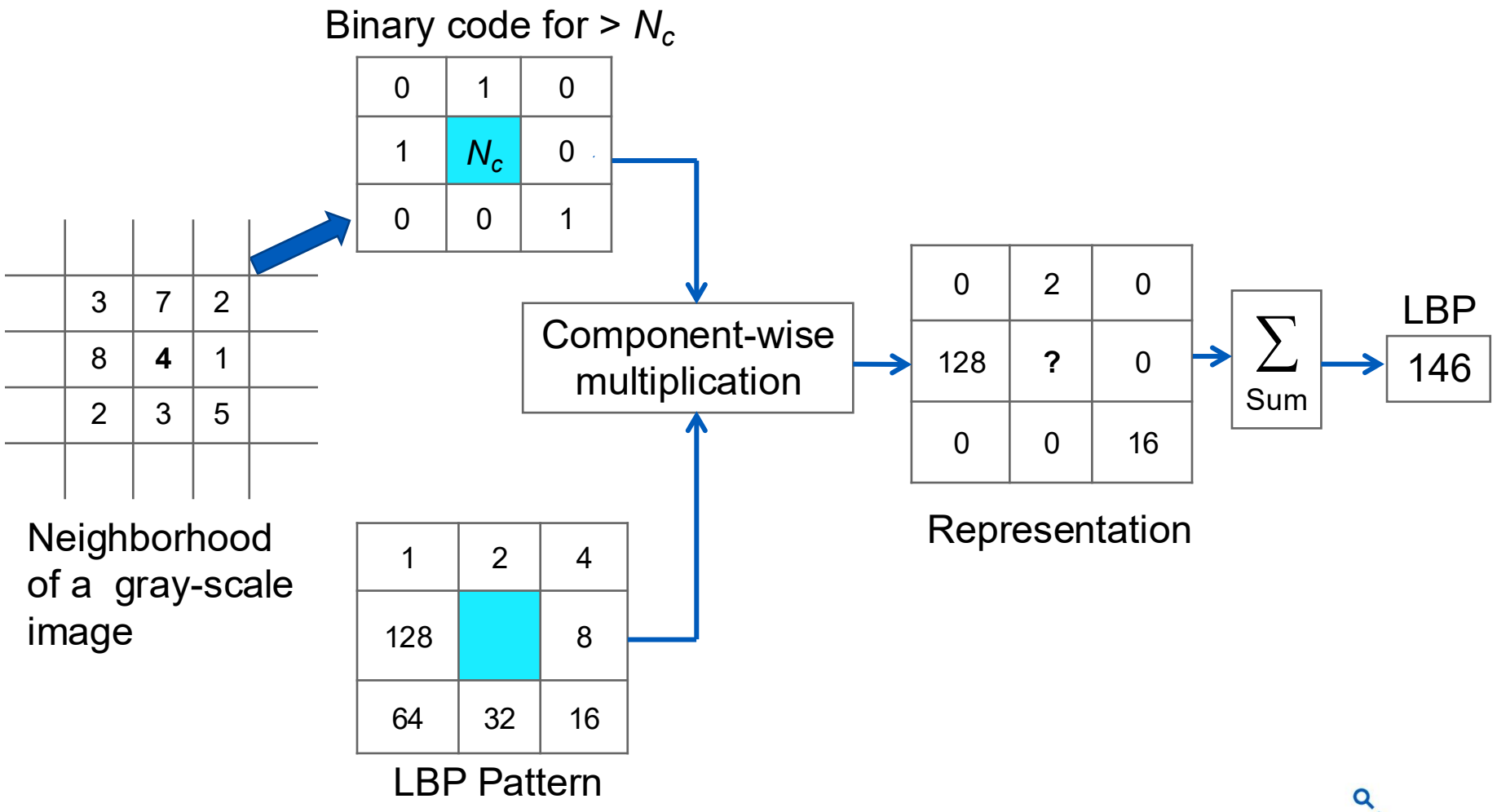
r → radius (for 3x3 cell, it is 1).

Binary threshold function $g(x)$ is,

$$g(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

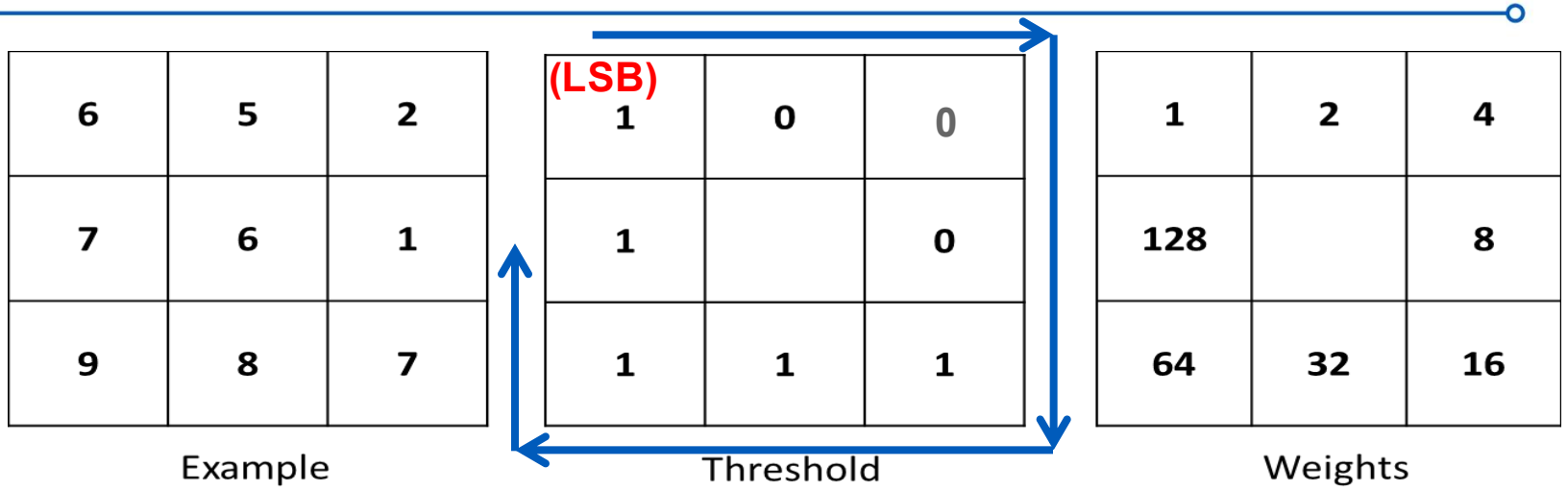


Computation of Local Binary Pattern



Example of how the *LBP operator* works

Computation of Local Binary Pattern



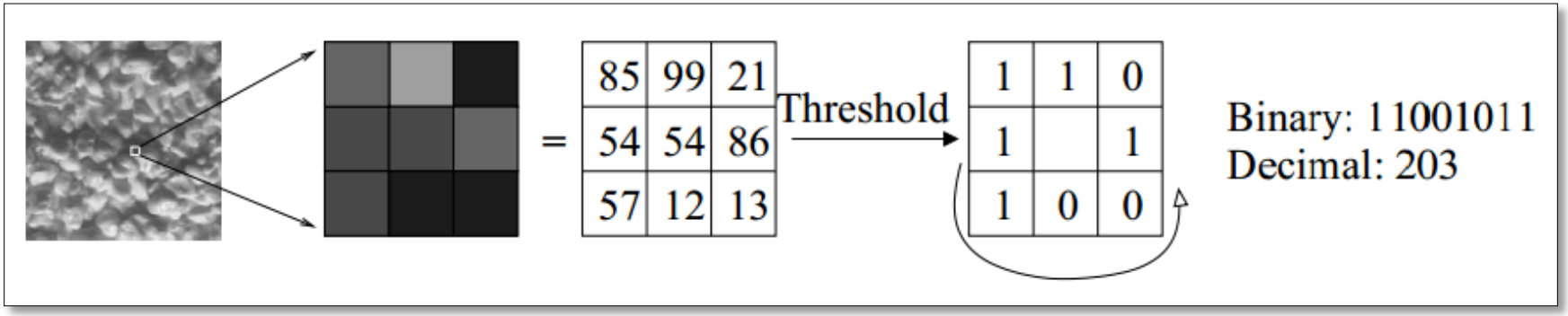
Binary Pattern:	1 (MSB)	1	1	1	0	0	0	1 (LSB)
Code/Weight (2^p):	1 x 2^7	1 x 2^6	1 x 2^5	1 x 2^4	0 x 2^3	0x 2^2	0x 2^1	1 x 2^0
	= 128	= 64	= 32	= 16	= 0	= 0	= 0	= 1

LBP:	128 + 64 + 32 + 16 + 0 + 0 + 0 + 1 = 241

MSB stands for Most Significant Bit, while **LSB** is Least Significant Bit.

LBP

- One of the best performing texture descriptors
- A label is assigned to every pixel
- Use center pixel to threshold the 3x3 neighborhood
  - Result in binary number
- Histogram of the labels is used as a texture descriptor



The Basic LBP operator

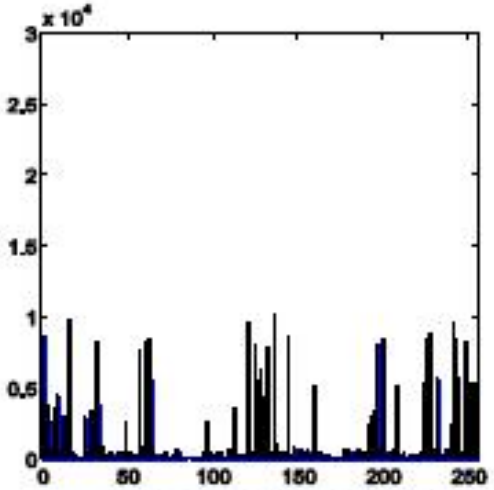
LBP hitogram



Input image



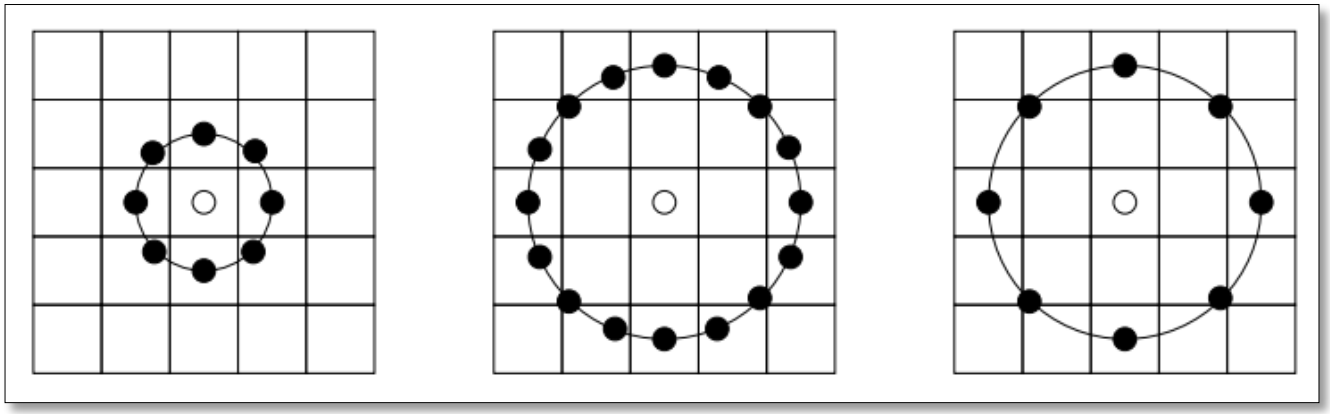
LBP image



LBP histogram

LBP

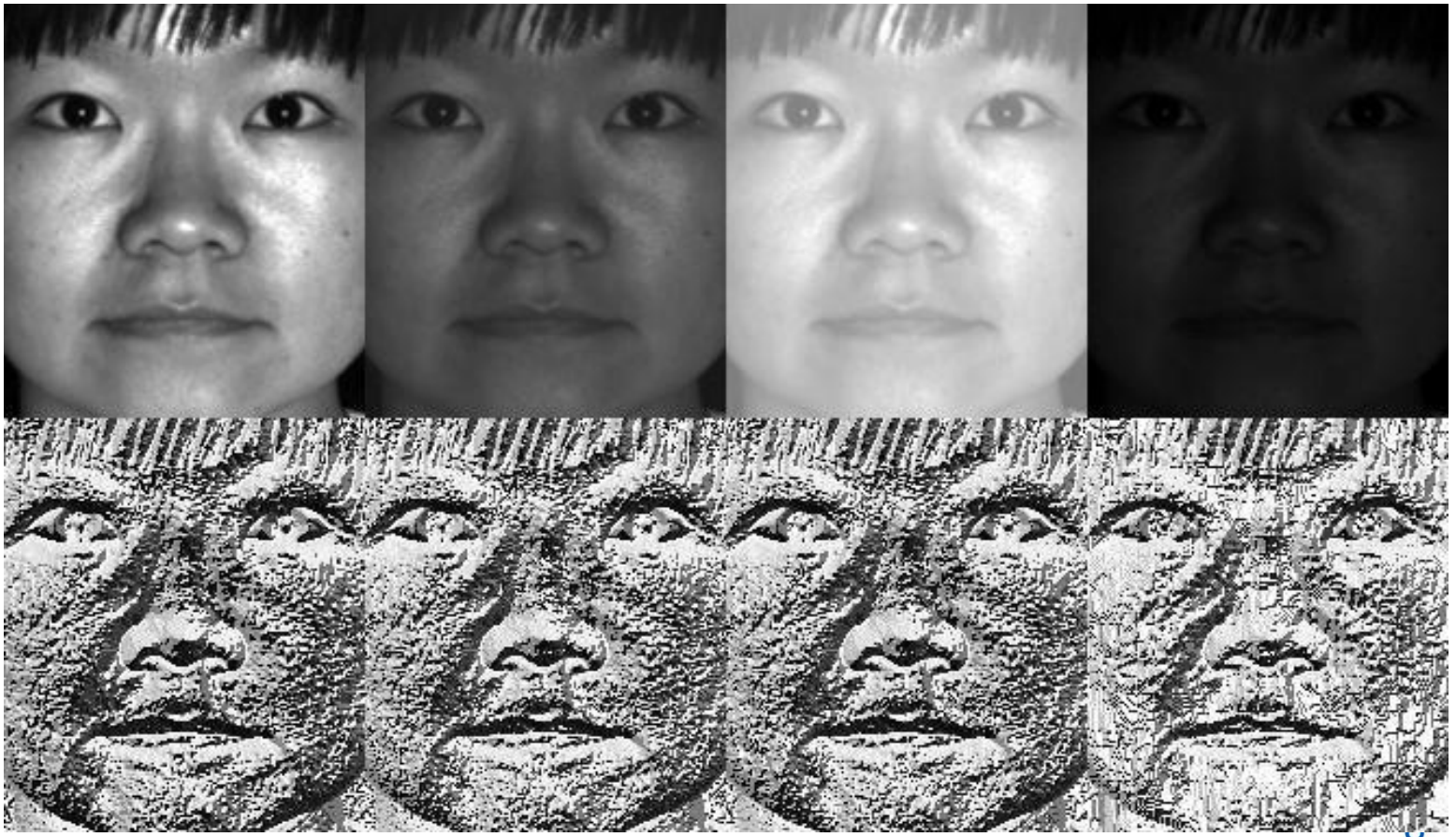
- LBP is extended to use different sizes of neighborhoods.
- Local neighborhoods is defined as a set of **sampling points**.
  - points evenly spaced on a circle centered at the labeled pixel.
- **(P, R) , P = number of sampling points, R = radius**
- **Bilinear interpolation** is used
  - If sampling point is not in the center of the pixel.



The circular (8,1), (16, 2) and (8, 2) neighborhoods.

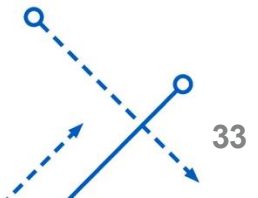
LBP Example

- Notice how LBP feature are illumination invariant



Uniform LBP

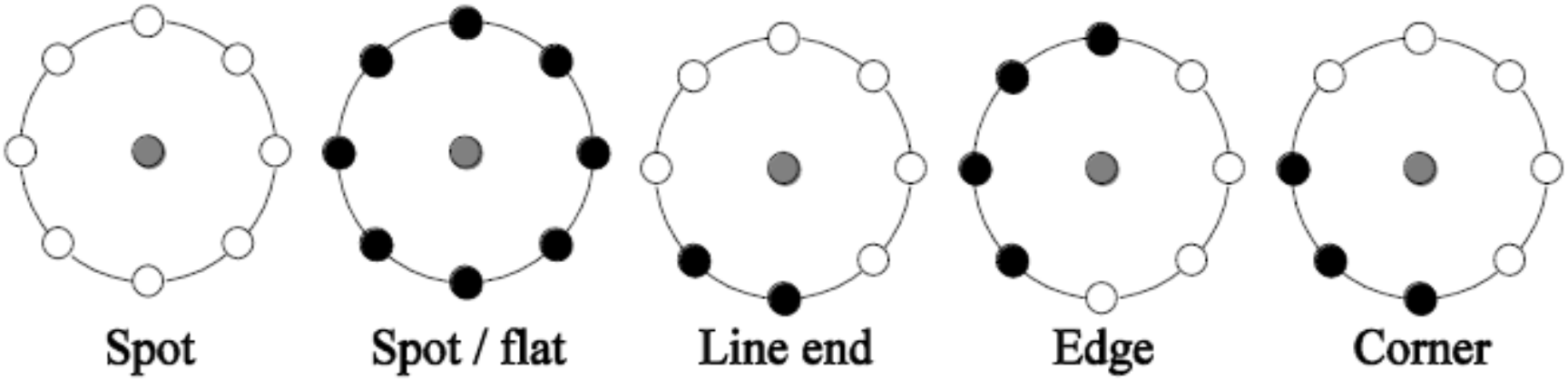
- Uniform patterns to further improve LBP.
  - Has at most 2 bitwise transitions in binary pattern.
  - Histogram
    - assigns separate bins for every uniform pattern.
    - assigns a single bin for all non-uniform pattern.



Uniform LBP

Texture primitives ("micro-textons") detected by the uniform patterns of LBP

1 = black
0 = white



Uniform LBP

- Uniform patterns examples
  - 00000000 (0 transitions)
  - 01110000 (2 transitions)
  - 11001111 (2 transitions)
- Non-uniform patterns examples
  - 11001001 (4 transitions)
  - 01010011 (5 transitions)
- 59 bins histogram
  - 58 uniform bins →
  - 1 non-uniform bin

Length of feature vector

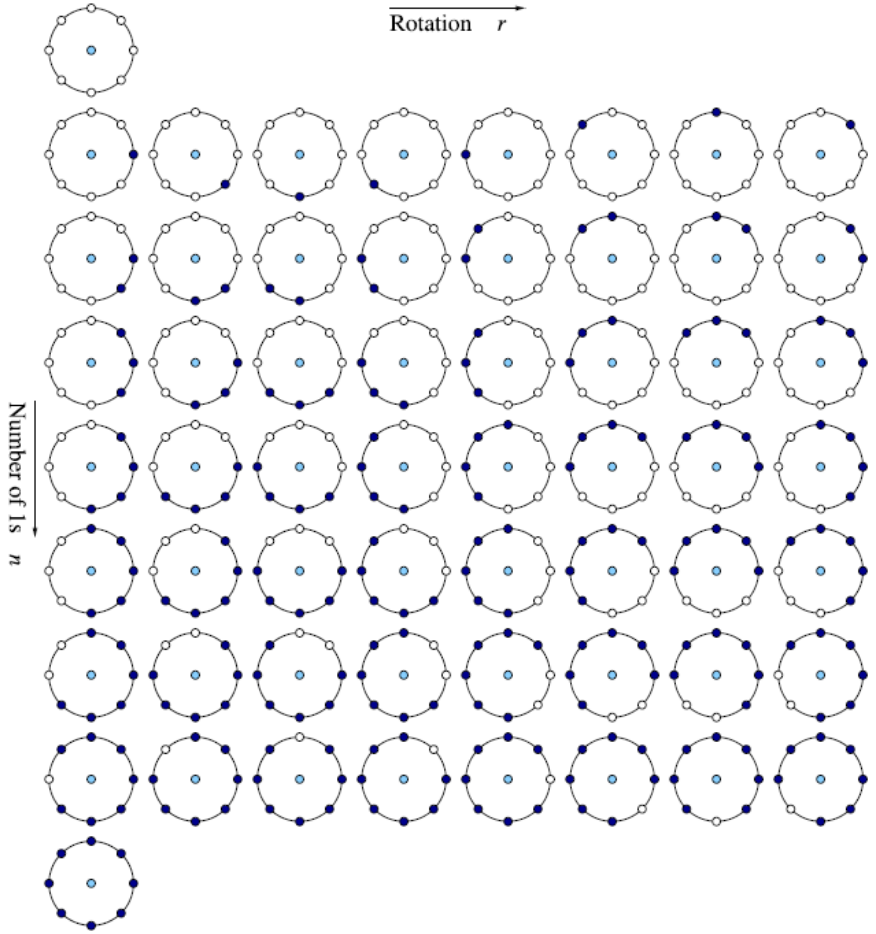


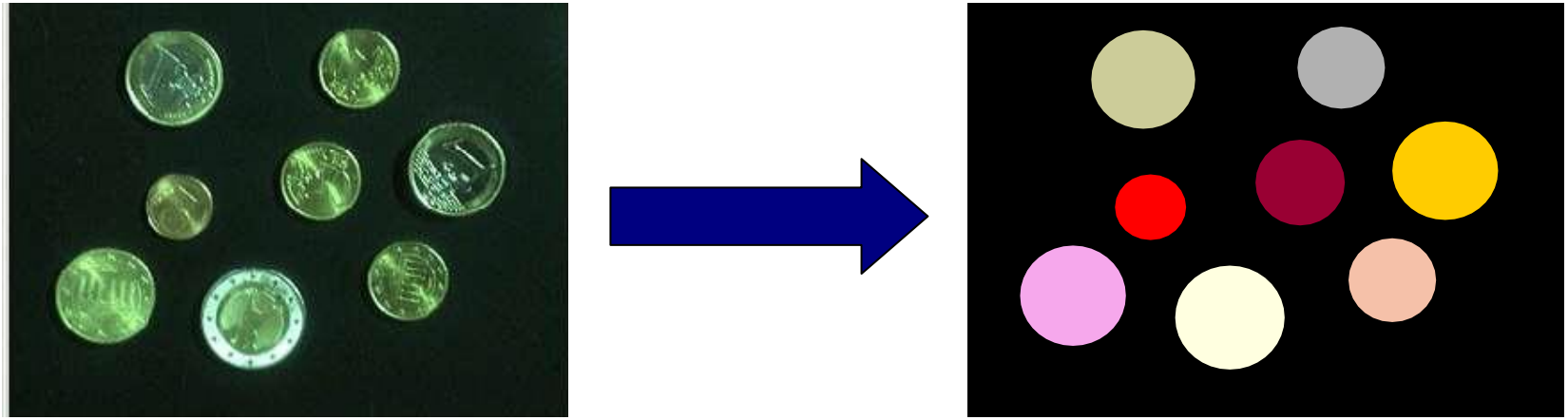
Fig. 2.4 The 58 different uniform patterns in (8, *R*) neighborhood

Other Measures of Texture

Texture measure	Description	Equation*
Mean	Mean is the average grey level for each sample	$\sum_{i,j=0}^{N-1} i(P_{i,j})$
Variance	Variance is a measure of heterogeneity; it increases when the grey level values differ from their mean. In general, coarse-textured features are associated with higher variances	$\sum_{i,j=0}^{N-1} P_{i,j}(i-\mu_i)^2$
Homogeneity	This parameter measures image homogeneity as it assumes larger values for smaller grey tone differences in pair elements	$\sum_{i,j=0.}^{N-1} \frac{P_{i,j}}{1+(i-j)^2}$
Contrast	Contrast is a measure of spatial frequency, the difference between the highest and the lowest values of a contiguous set of pixels. A high contrast implies high coarse texture	$\sum_{i,j=0}^{N-1} P_{i,j}(i-j)^2$
Dissimilarity	Dissimilarity, akin to contrast, describes the heterogeneity of the grey levels. Higher values of dissimilarity in the GLC matrix indicate coarser textures	$\sum_{i,j=0}^{N-1} P_{i,j}\lvert i-j\rvert$
Entropy	Entropy measures the disorder of an image. When the image is not texturally uniform, many GLC matrix elements have very small values, which imply that entropy is very large	$\sum_{i,j=0}^{N-1} P_{i,j}(-\ln p_{i,j})^2$
Angular second moment	This parameter measures textural uniformity. Thus, high angular second moment values occur when the grey level distribution over the window has either a constant or a periodic form	$\sum_{i,j=0}^{N-1} {P_{i,j}}^2$
Correlation	Correlation is a measure of grey-tone linear dependencies in the image. High correlation values imply a linear relationship between the grey levels of pixel pairs	$\sum_{i,j=0}^{N-1} P_{i,j}\left[\frac{(i-\mu_i)(j}{\sqrt{(\mathrm{SD}_i^2)}}\right.$

Image Segmentation

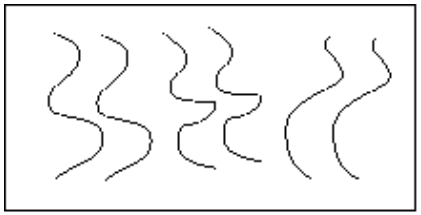
- **Partition** the pixels of an image into **groups** that strongly correlate with the objects in an image
- The first step in many automated computer vision applications.



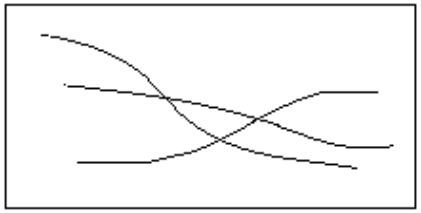
Segmentation

Segmentation algorithms generally are based on one of two basis properties of intensity values.

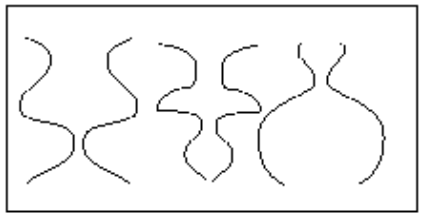
- **Discontinuity:** to partition an image based on abrupt changes in intensity (such as edges).
- **Similarity:** to partition an image into regions that are similar according to a set of predefined criteria.



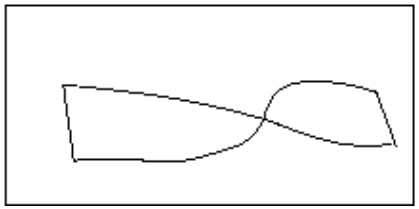
Parallelism



Continuity



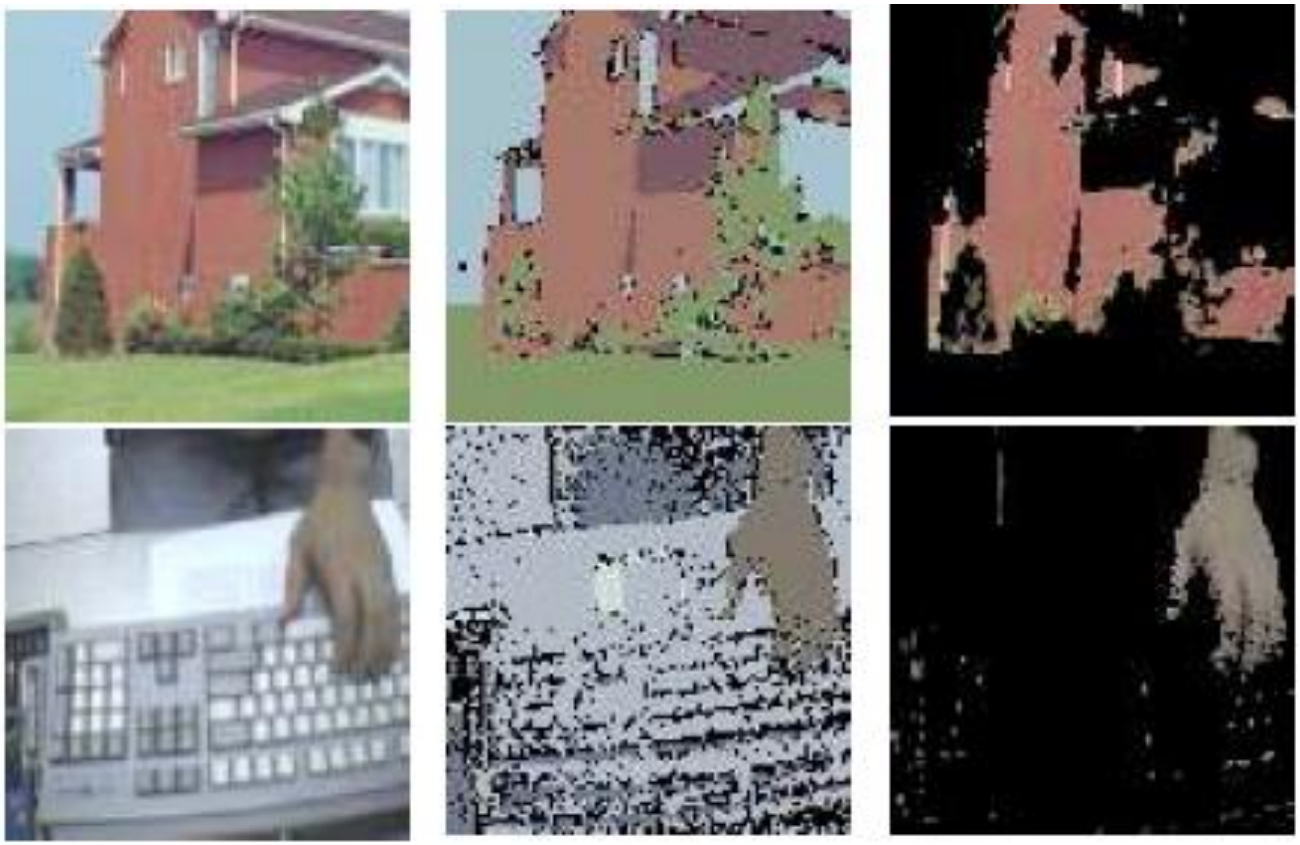
Symmetry



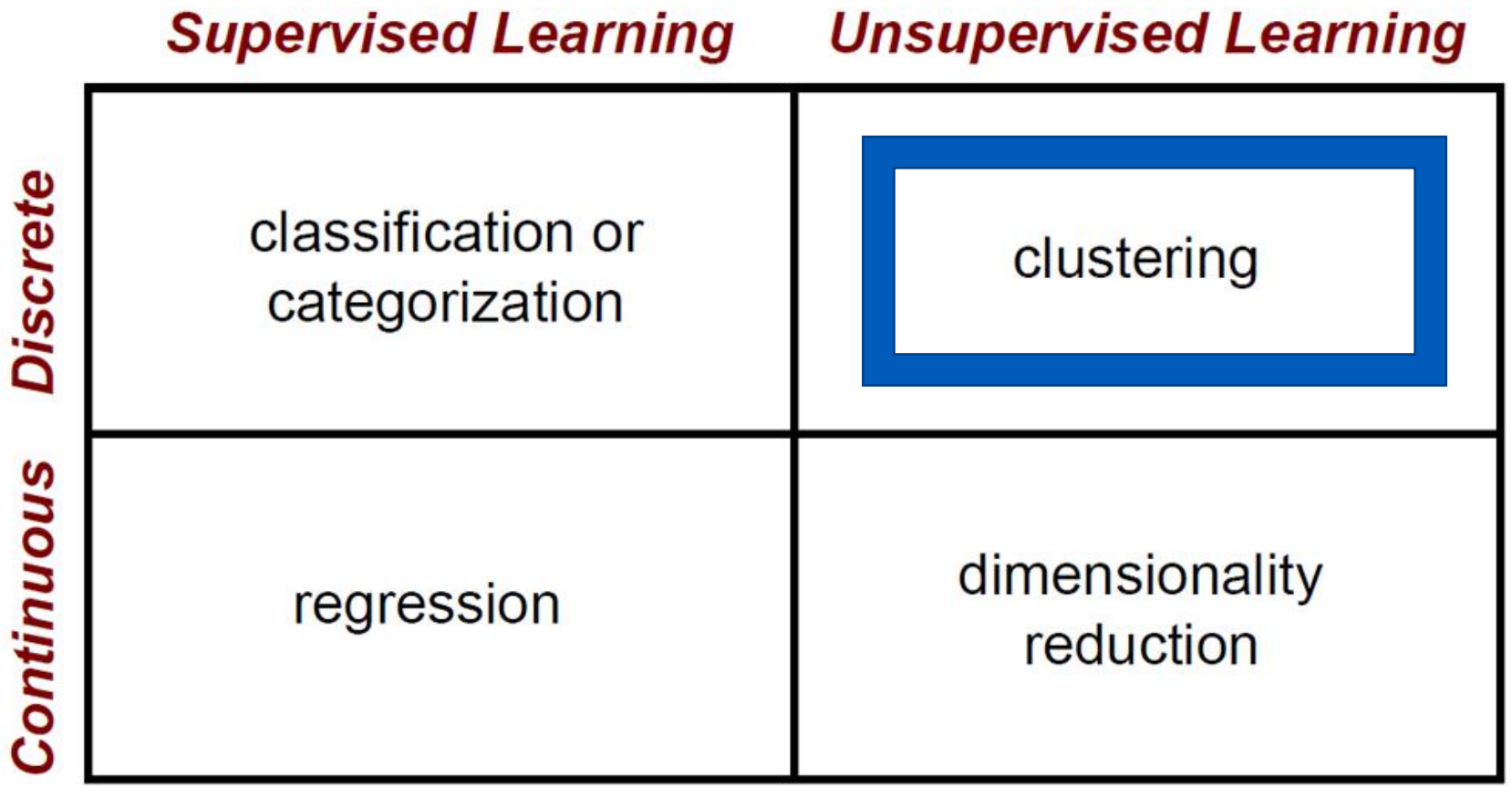
Closure

Segmentation needs Grouping/Clustering

- Grouping (or clustering)
  - Collect pixels that “belong together”



Machine Learning



Clustering example: image segmentation

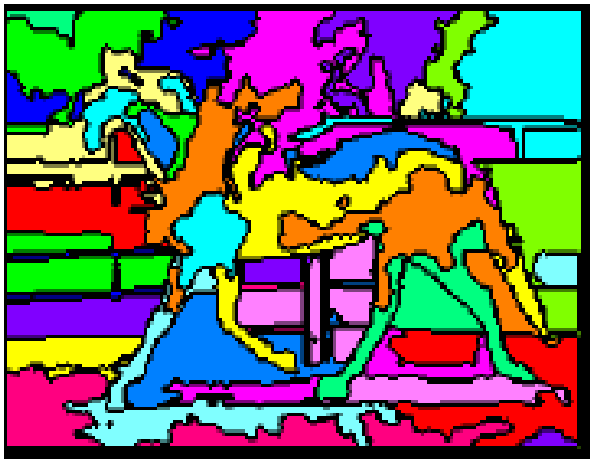
- Break up the image into **meaningful** or **perceptually similar** regions

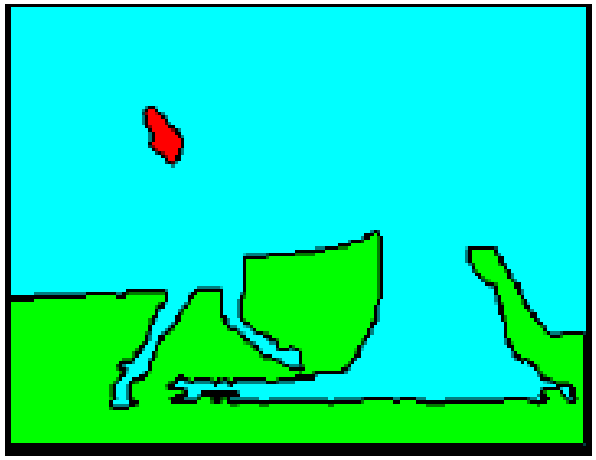


Types of segmentations

Oversegmentation

Undersegmentation

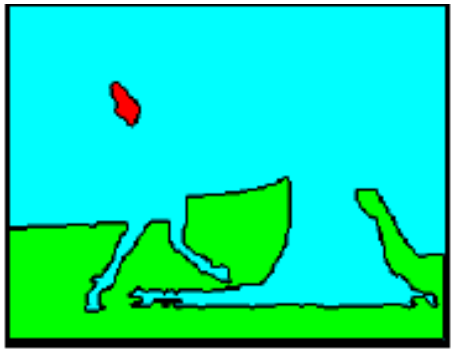










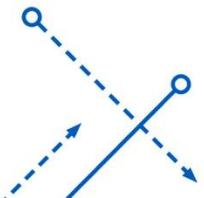


Multiple Segmentations

Clustering

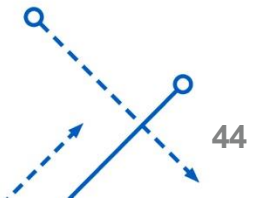
- Group together similar points and represent them with a single category.

- Key Challenges:
  - What makes two points/images/patches similar?
  - How do we compute an overall grouping from pairwise similarities?



How do we cluster?

- K-means
  - Iteratively re-assign points to nearest cluster center
- Single-link clustering
  - Start with each point as its own cluster and iteratively merge the closest clusters
- Mean-shift clustering
  - Estimate modes of PDF



Clustering for Summarization

- **A cluster**: a "center" (in feature space) and a list of data points it contains.
- Goal:
  - Minimize variance in data given clusters
  - Preserve information

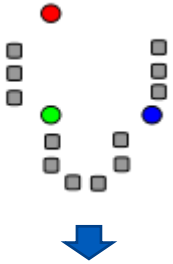
Cluster Center Data

$$\mathbf{c}^*, \boldsymbol{\delta}^* = \underset{\mathbf{c},\boldsymbol{\delta}}{\operatorname{argmin}} \frac{1}{N} \sum_{j}^{N} \sum_{i}^{K} \delta_{ij} \left(\mathbf{c}_i - \mathbf{x}_j \right)^2$$

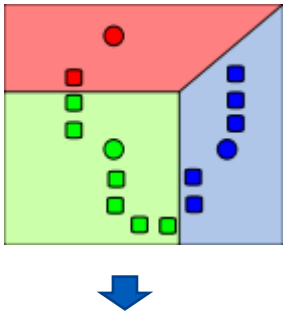
Whether x_j is assigned to c_i

K-means algorithm

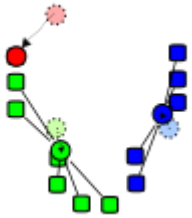
1. Randomly select K centers (means)



2. Assign each point to nearest center (mean)

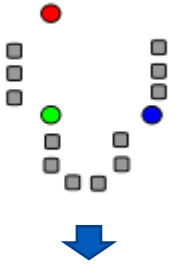


3. Compute new center (mean) for each cluster

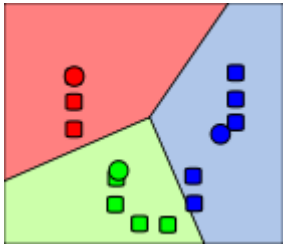


K-means algorithm

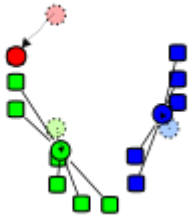
1. Randomly select K centers



2. Assign each point to nearest center



3. Compute new center (mean) for each cluster



Back to 2

Illustration: http://en.wikipedia.org/wiki/K-means_clustering

K-means algorithm

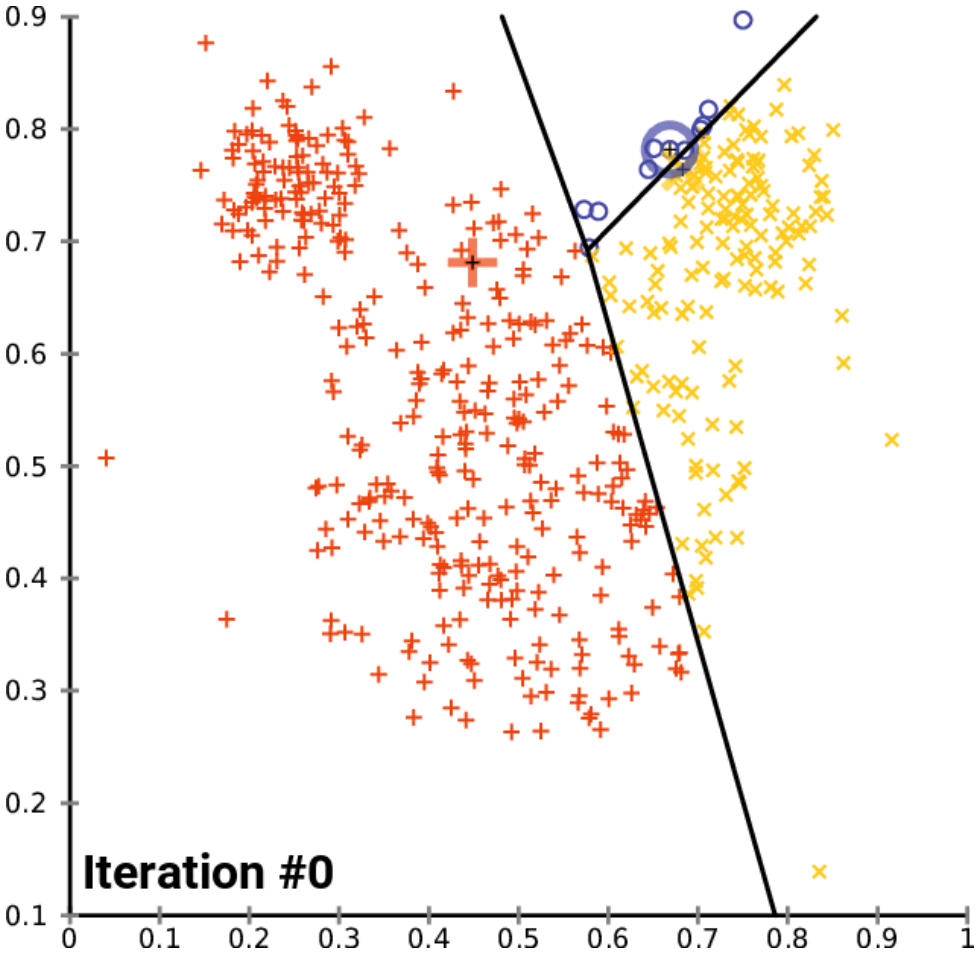


Illustration: http://en.wikipedia.org/wiki/K-means_clustering

K-means algorithm

1. Initialize cluster centers: $\mathbf{c}^0$; t=0

2. Assign each point to the closest center

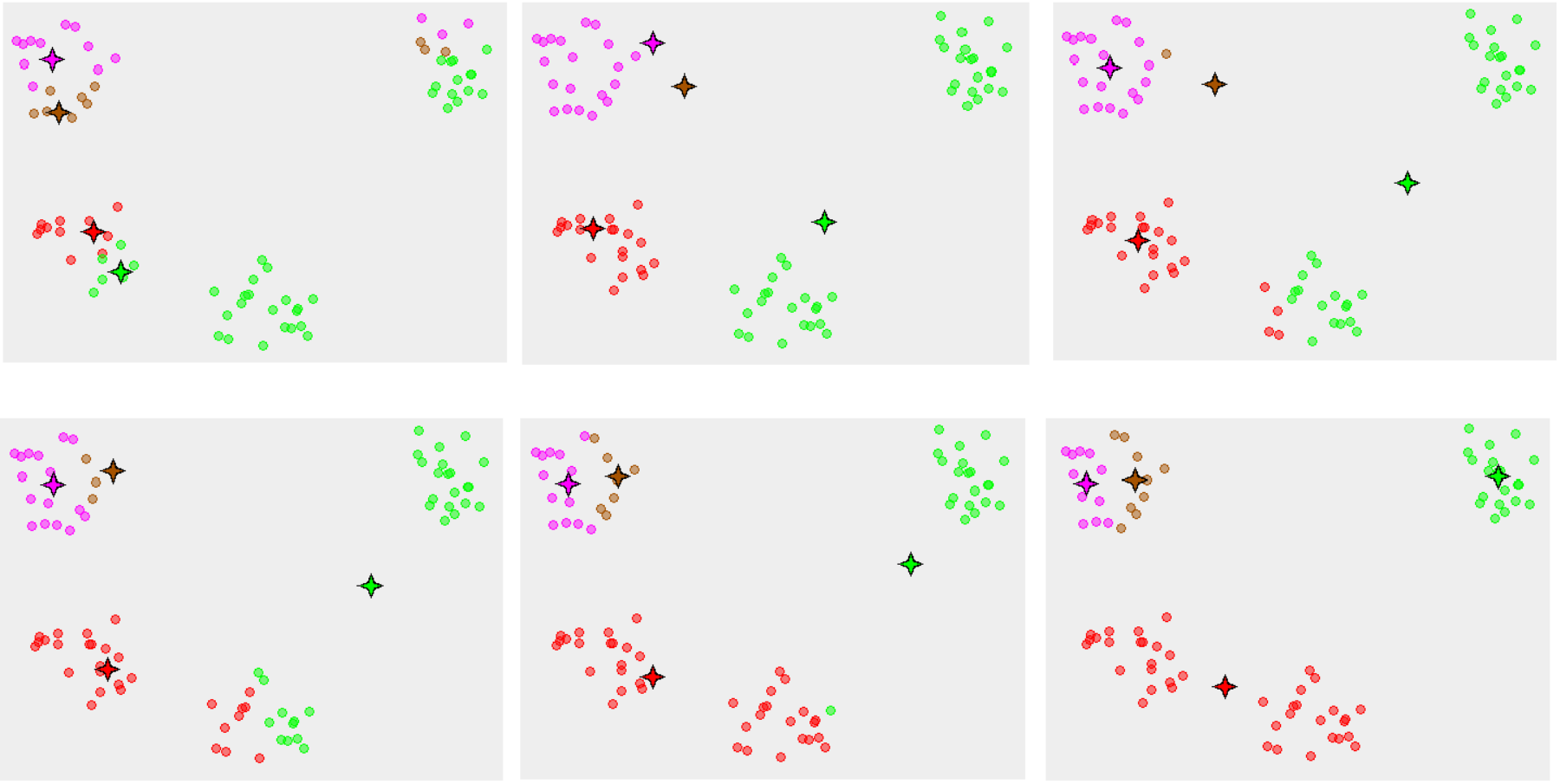
$$\boldsymbol{\delta}^t = \underset{\boldsymbol{\delta}}{\operatorname{argmin}} \frac{1}{N} \sum_j^N \sum_i^K \delta_{ij} \left(\mathbf{c}_i^{t-1} - \mathbf{x}_j \right)^2$$

3. Update cluster centers as the mean of the points

$$\mathbf{c}^t = \underset{\mathbf{c}}{\operatorname{argmin}} \frac{1}{N} \sum_j^N \sum_i^K \delta_{ij}^t \left(\mathbf{c}_i - \mathbf{x}_j \right)^2$$

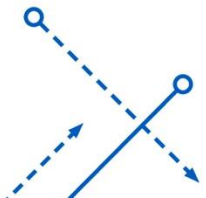
4. Repeat 2-3 until no points are re-assigned (t=t+1)

K-means may converge to local minimum



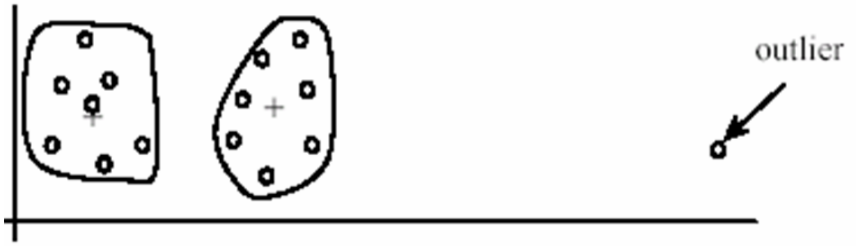
K-means: design choices

- Initialization
  - Randomly select K points as initial cluster center
  - Or greedily choose K points to minimize residual
- Distance measures
  - Traditionally Euclidean, could be others
- Optimization
  - May converge to a *local minimum*
  - May want to perform **multiple restarts**
    - **Combine with RANSAC Algorithm**

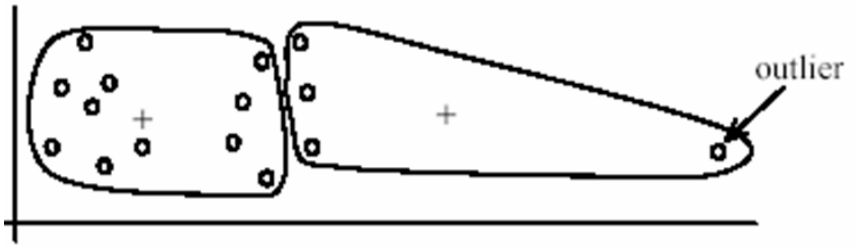


K-Means pros and cons

- Pros
  - Finds cluster centers that minimize conditional variance
    - good representation of data
  - Simple and fast*
  - Easy to implement
- Cons
  - Need to choose K
  - Sensitive to outliers
  - Prone to local minima
  - All clusters have the same parameters (e.g., distance measure is non-adaptive)
  - *Can be slow: each iteration is O(KNd) for N d-dimensional points
- Usage
  - Rarely used for pixel segmentation

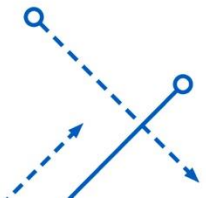


(B): Ideal clusters

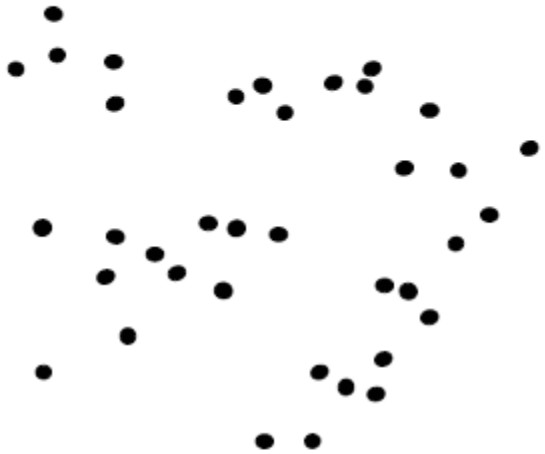


How do we cluster?

- K-means
  - Iteratively re-assign points to the nearest cluster center
- Single-link clustering
  - Start with each point as its own cluster and iteratively merge the closest clusters
- Mean-shift clustering
  - Estimate modes of PDF

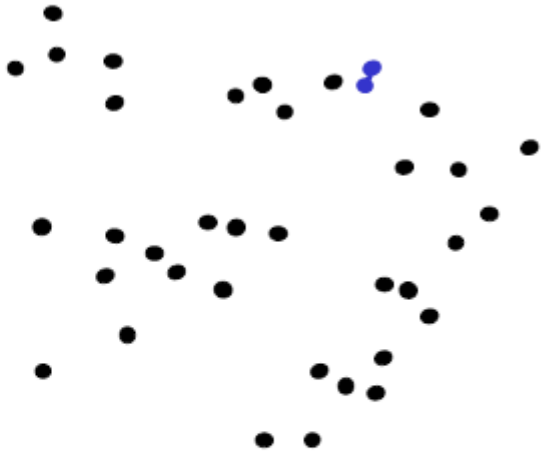


Single Link Clustering



1. Say "Every point is its own cluster"

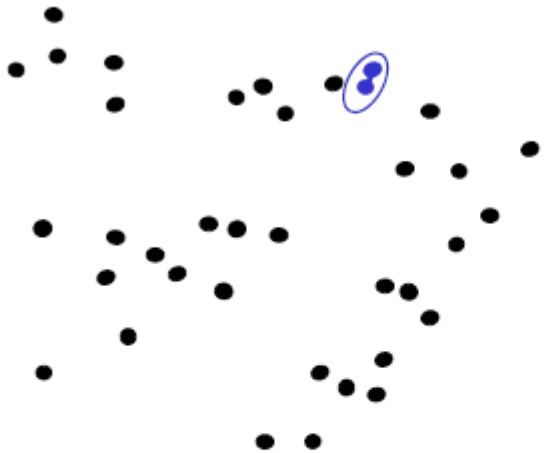
Single Link Clustering



1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters



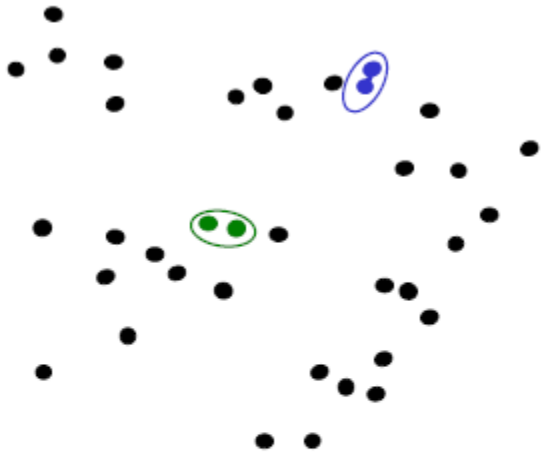
Single Link Clustering



1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent cluster



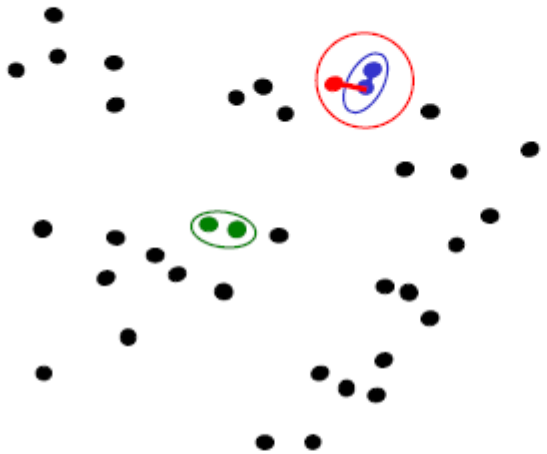
Single Link Clustering



1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent cluster
4. Repeat



Single Link Clustering

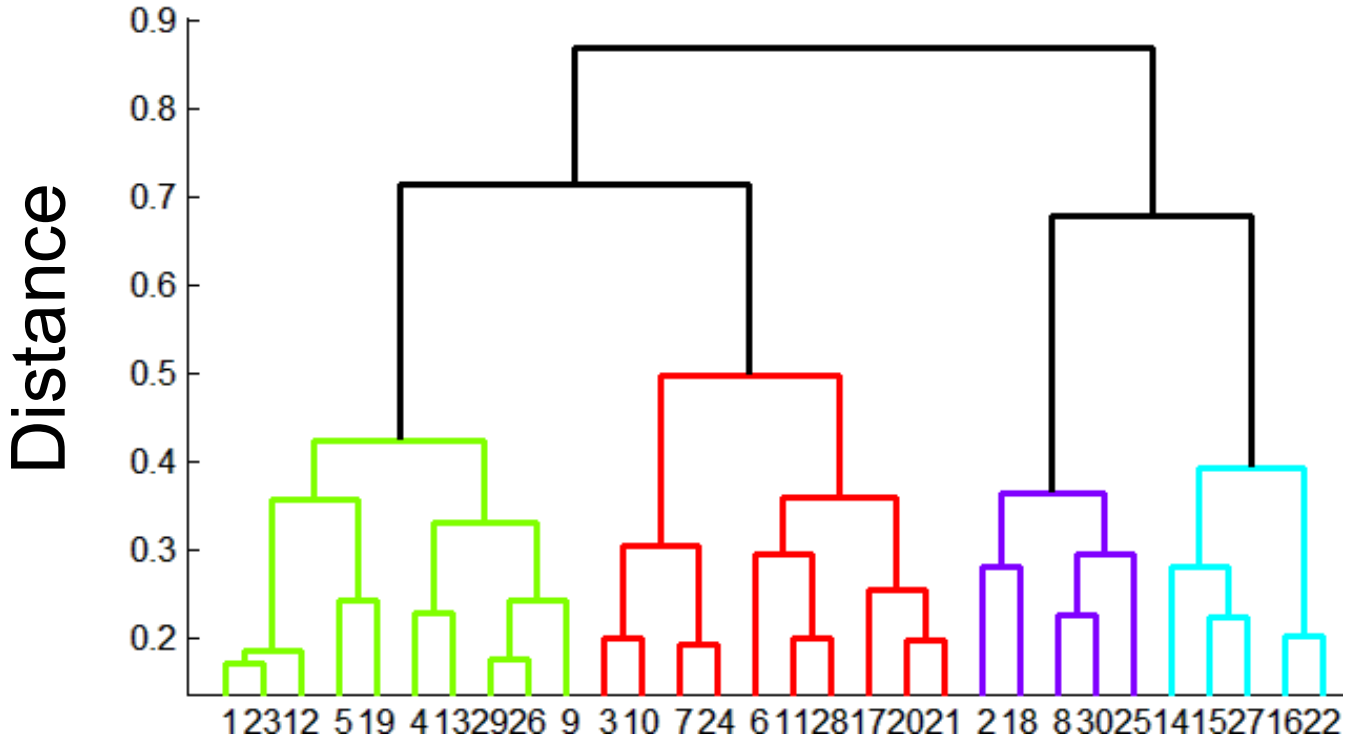


1. Say "Every point is its own cluster"
2. Find "most similar" pair of clusters
3. Merge it into a parent cluster
4. Repeat



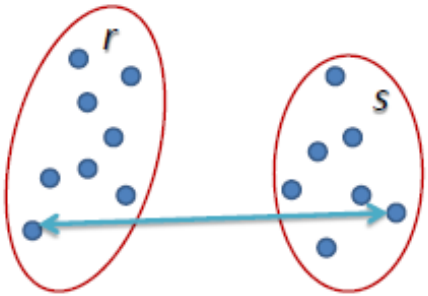
Single Link Clustering

- How many clusters?
  - Threshold based on max number of clusters or based on distance between merges

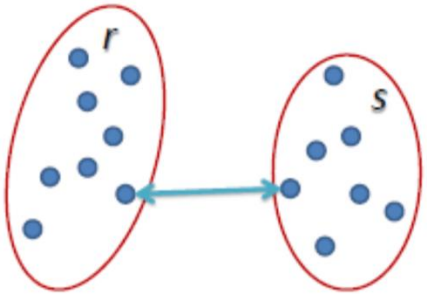


Single Link Clustering

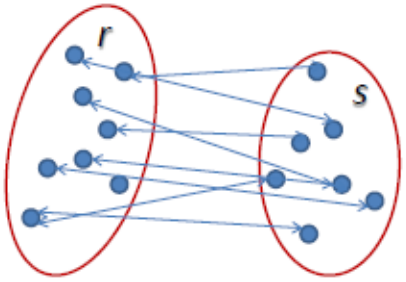
- How to define cluster similarity?
  - Average/maximum/minimum distance of points
  - Distance between means.



$$L(r,s) = \max(D(x_{ri}, x_{sj}))$$



$$L(r,s) = \min(D(x_{ri}, x_{sj}))$$



$$L(r,s) = \frac{1}{n_r n_s} \sum_{i=1}^{n_r} \sum_{j=1}^{n_s} D(x_{ri}, x_{sj})$$

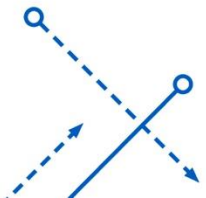
Summary: Single Link Clustering

Good

- Simple to implement, widespread application
- Clusters have adaptive shapes
- Provides a hierarchy of clusters

Bad

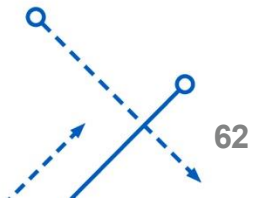
- May have **imbalanced** clusters
- Still have to choose number of clusters or threshold
- Need to use an “ultrametric” to get a meaningful hierarchy



How do we cluster?

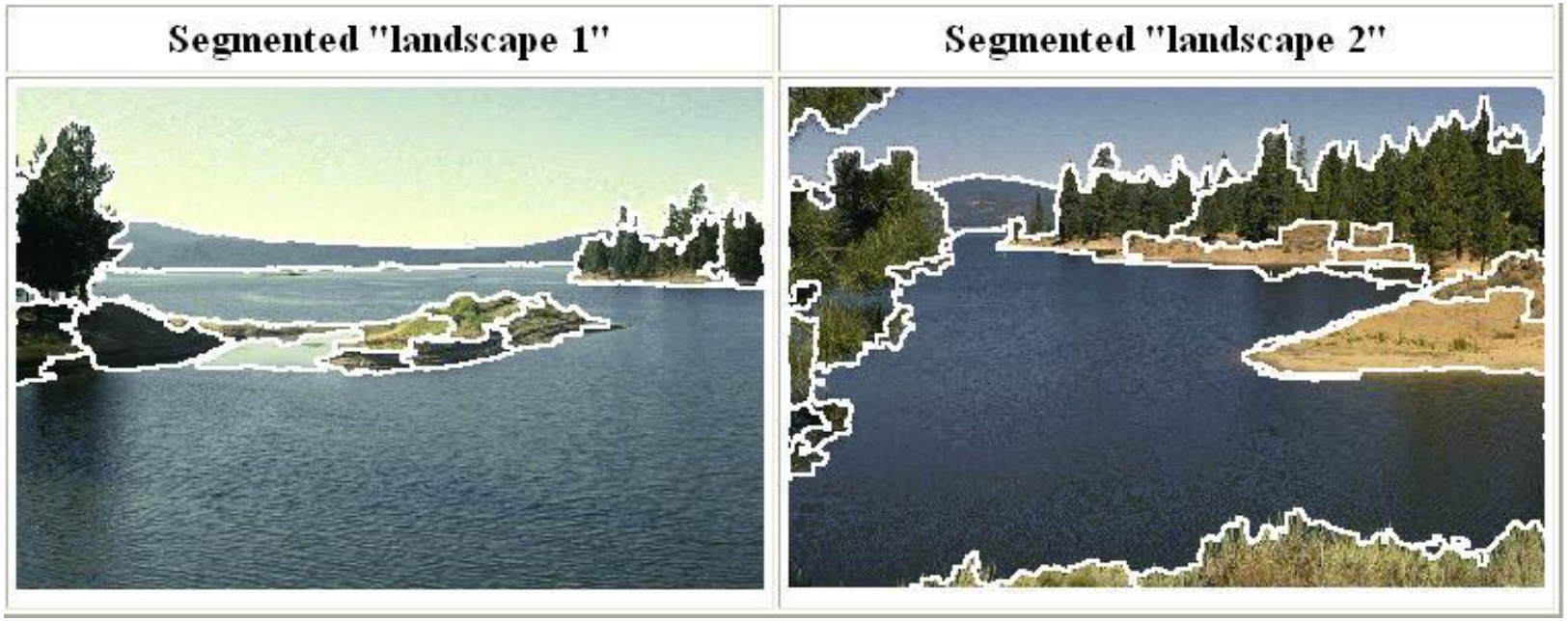
- K-means
  - Iteratively re-assign points to the nearest cluster center
- Single-link clustering
  - Start with each point as its own cluster and iteratively merge the closest clusters
- Mean-shift clustering

  –Estimate modes of PDF



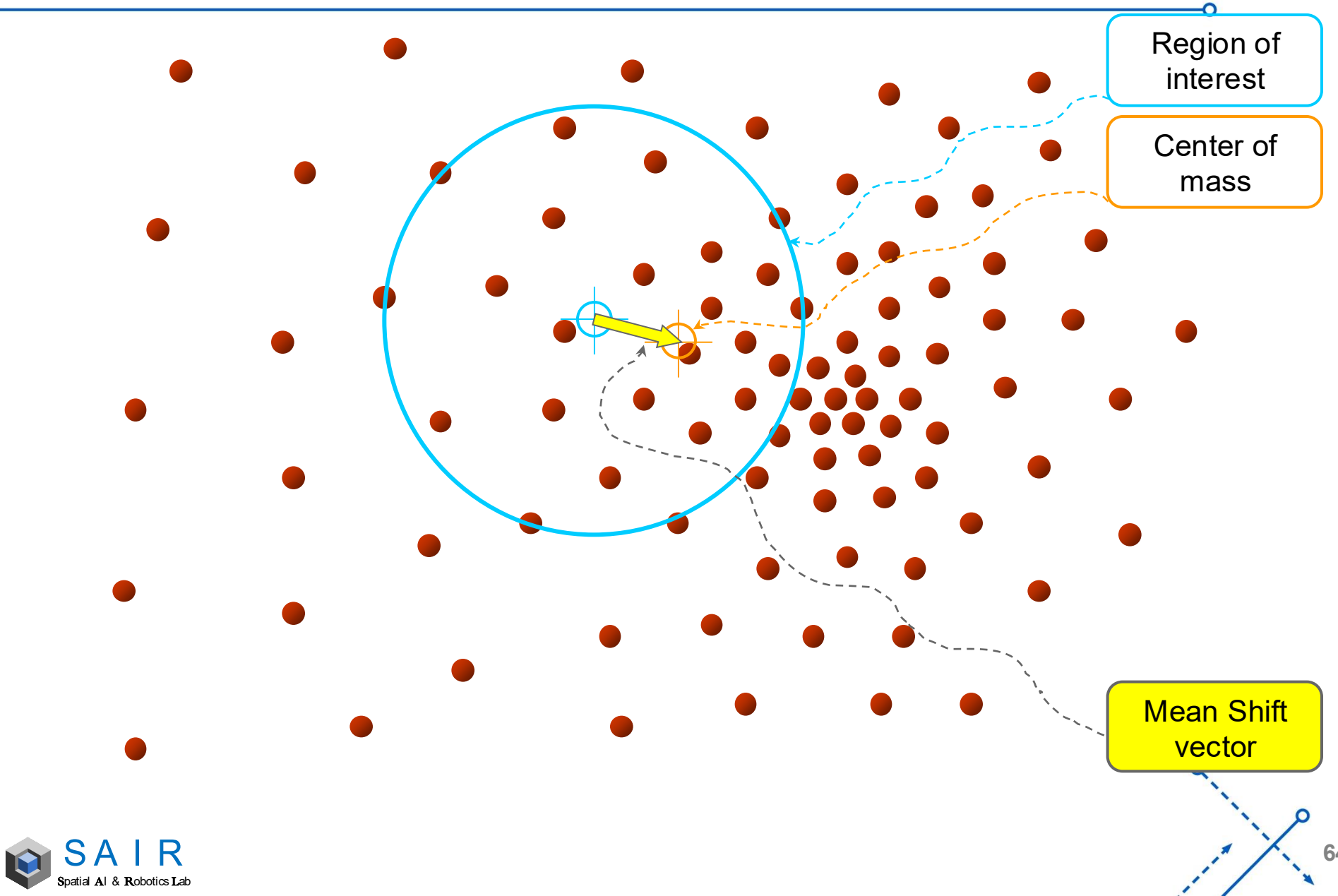
Mean shift algorithm

- Non-parametric feature-space analysis for locating the maxima of a density function.
- Versatile technique for clustering-based segmentation.

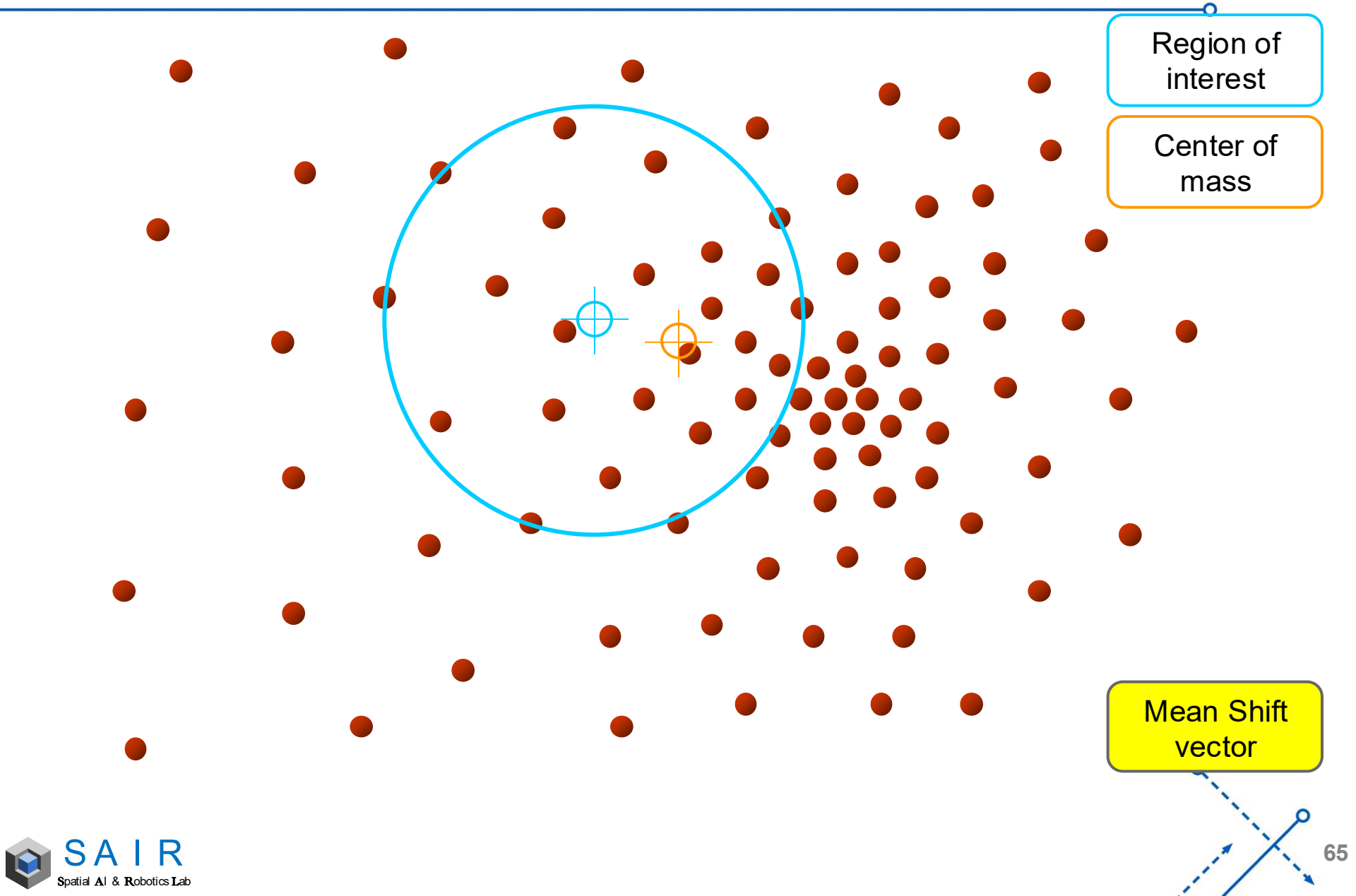


D. Comaniciu and P. Meer, Mean Shift: A Robust Approach toward Feature Space Analysis, PAMI 2002

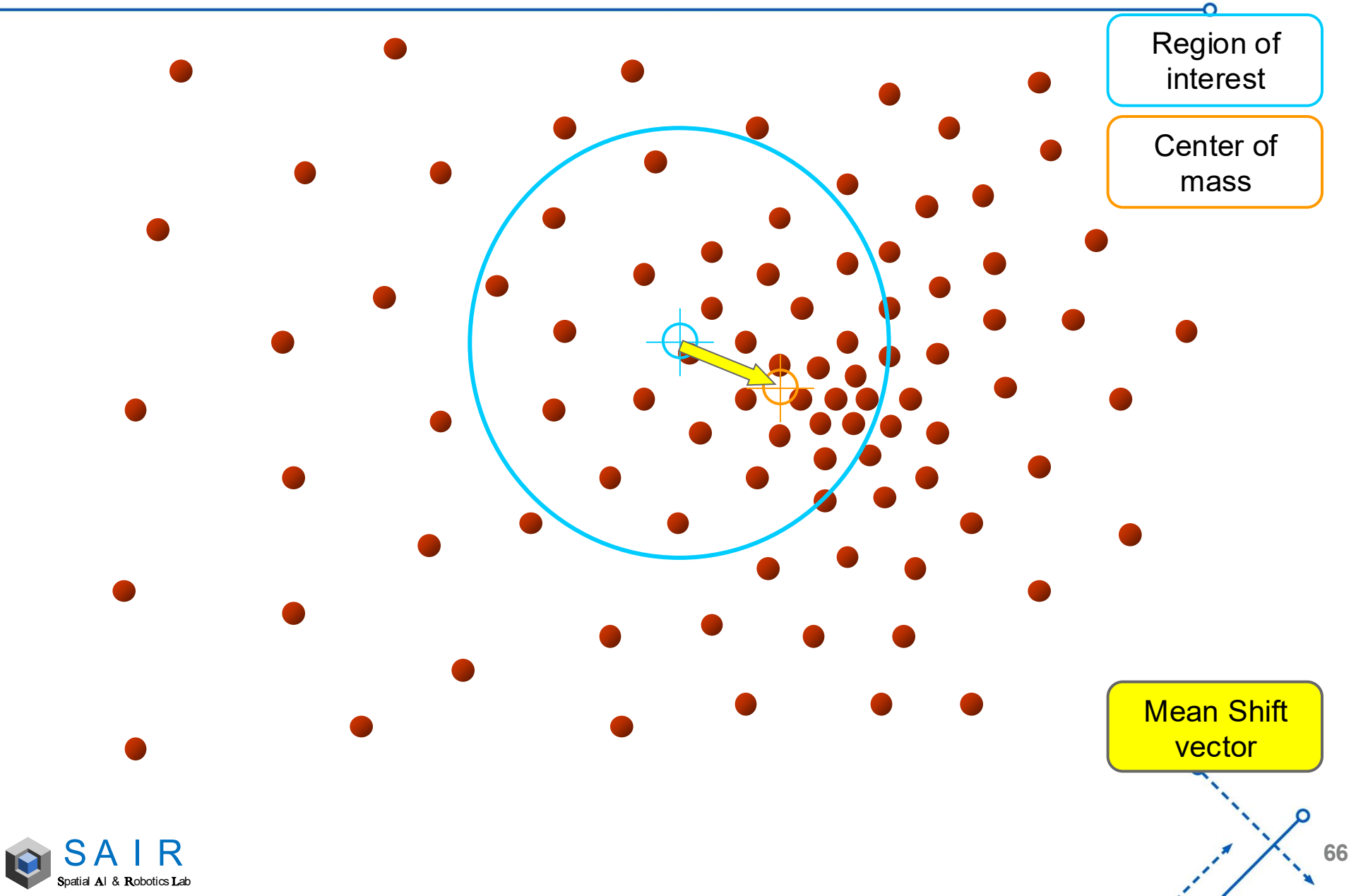
Mean shift



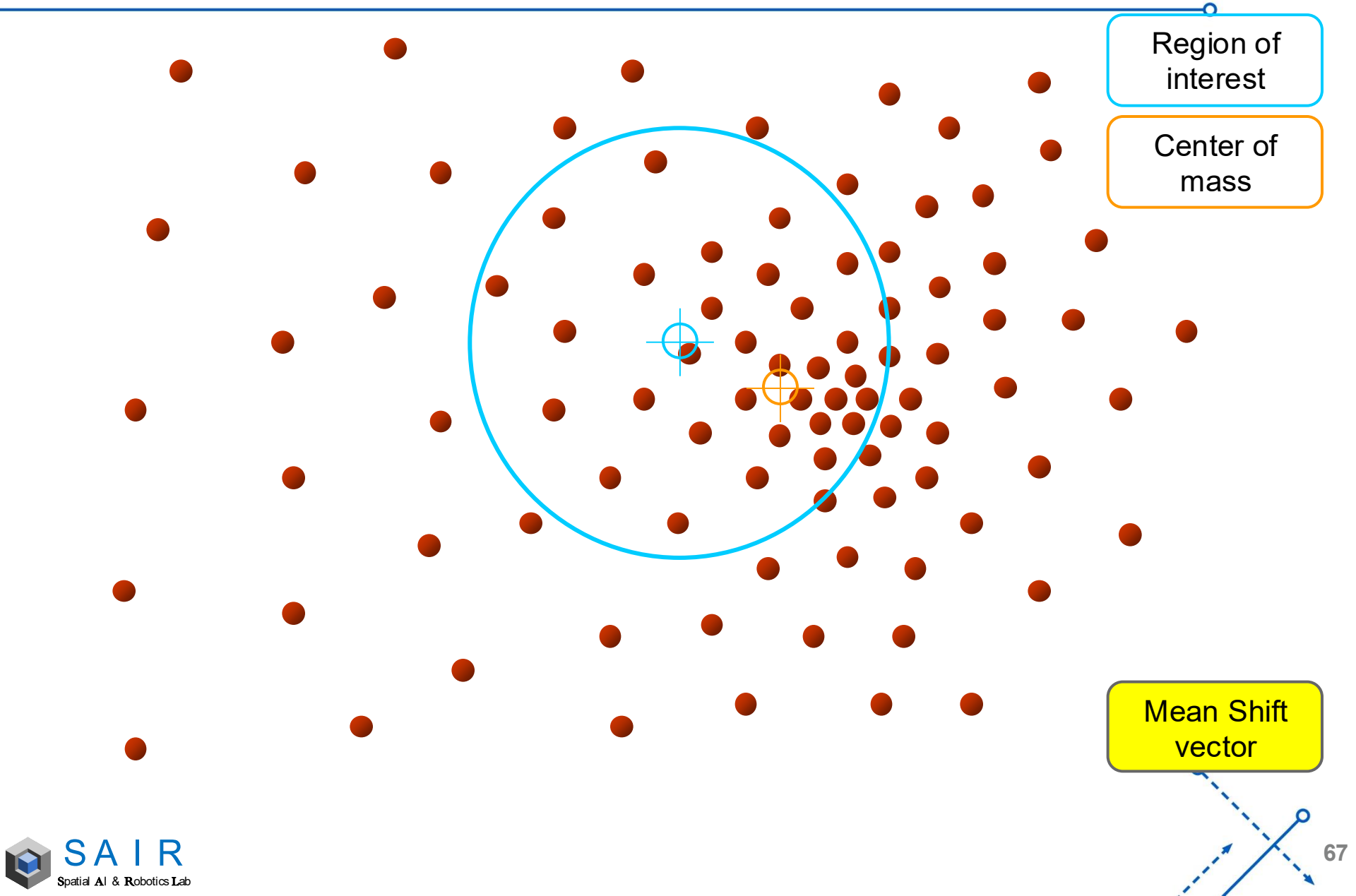
Mean shift



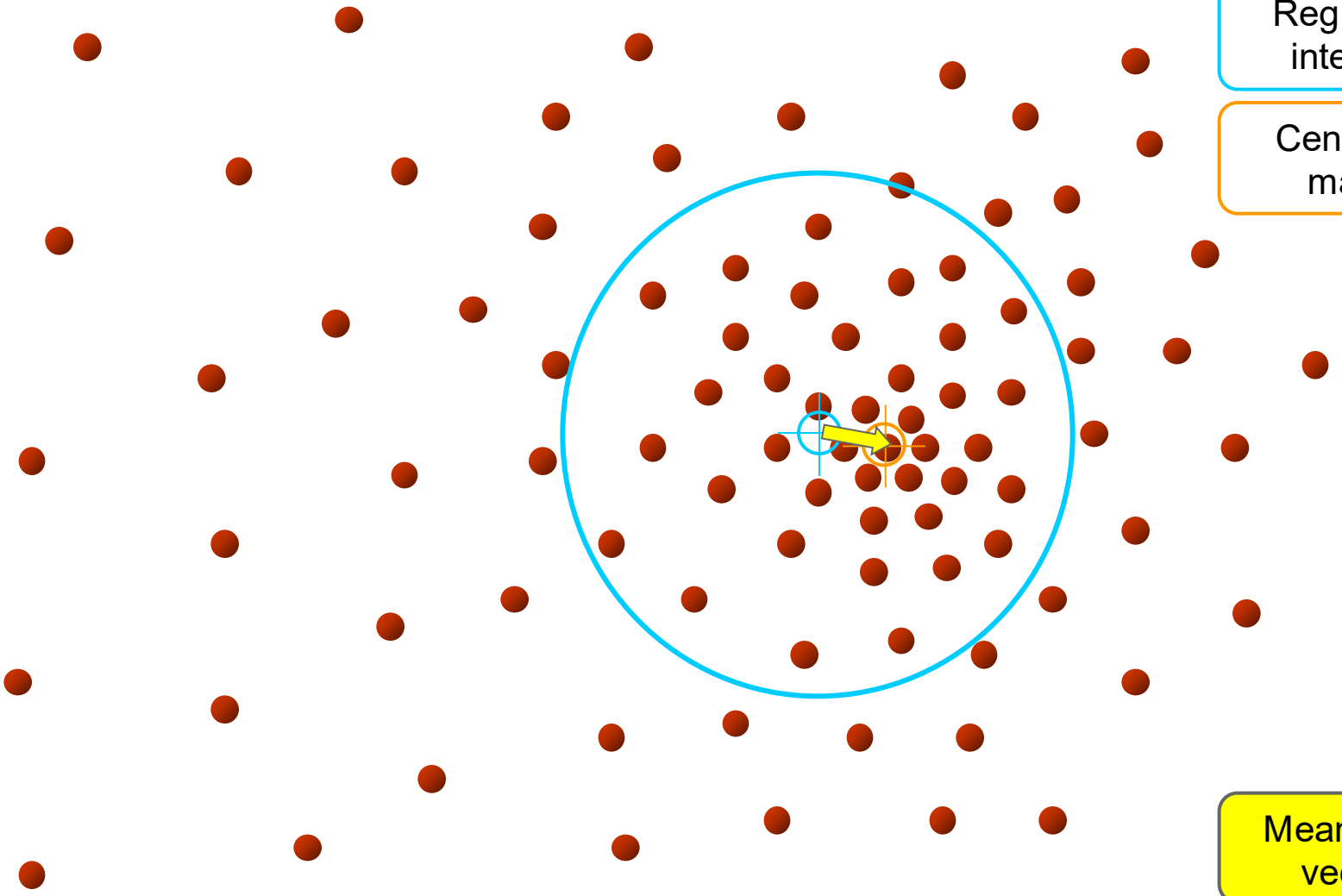
Mean shift



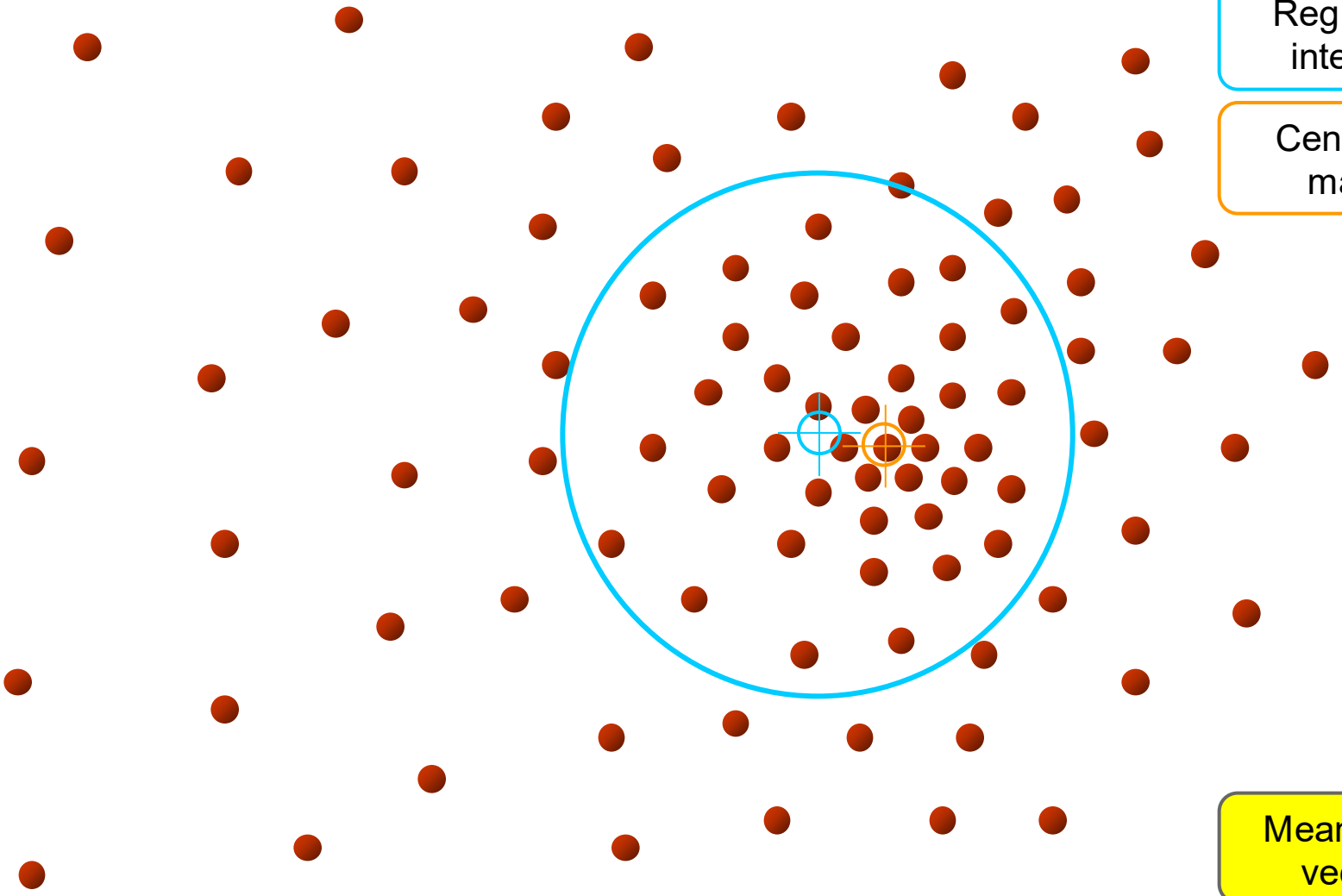
Mean shift



Mean shift

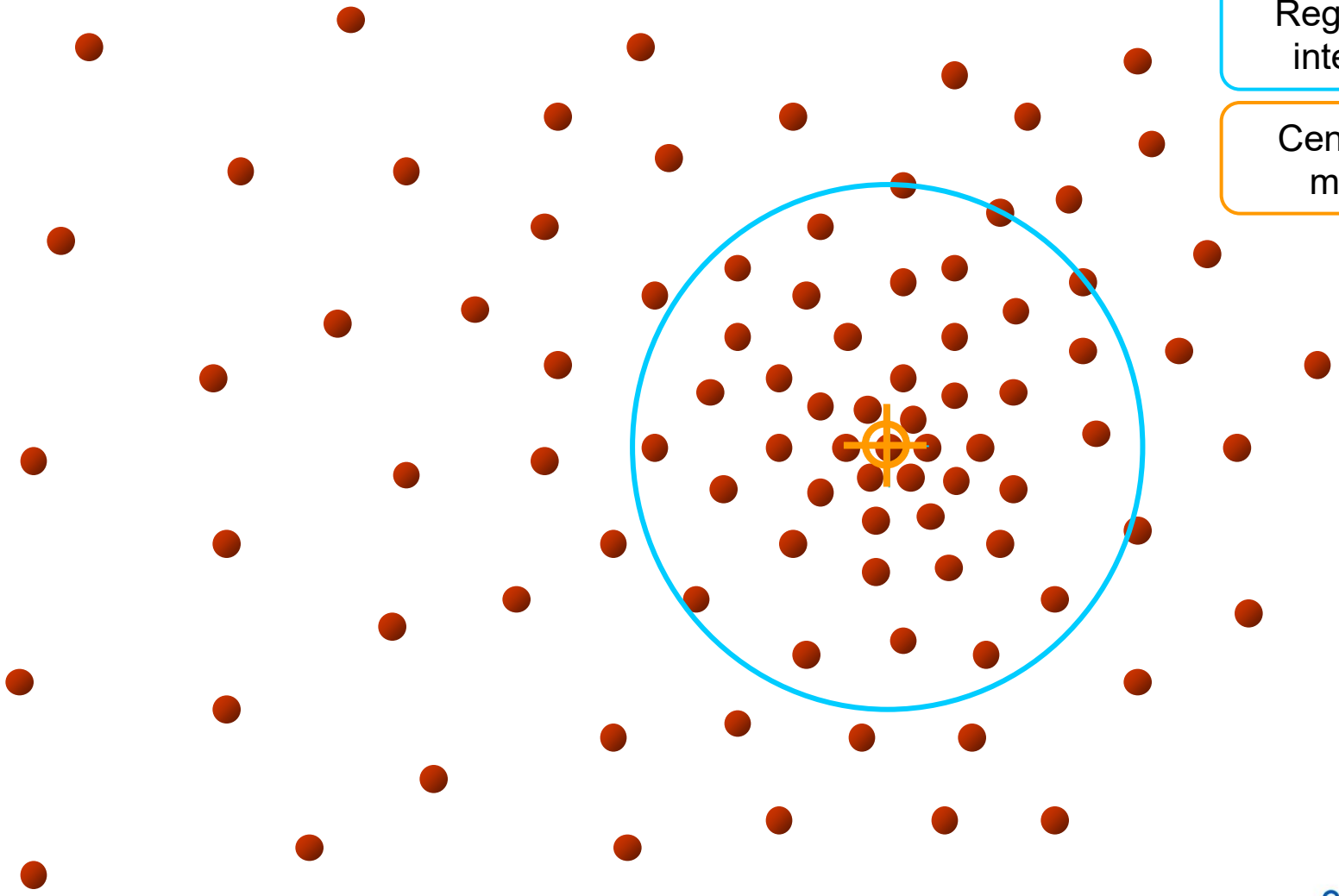


Mean shift



Slide by Y. Ukrainitz & B. Sarel

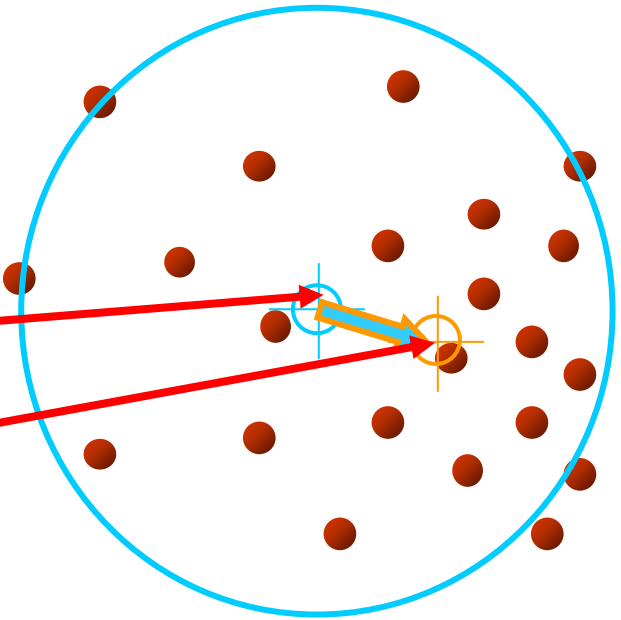
Mean shift



Computing the Mean Shift

- Compute mean shift vector.
- Translate mean by m(x), weighted by kernel function.

$$\mathbf{m}(\mathbf{x}) = \left[\frac{\sum_{i=1}^{n} \mathbf{x}_i g\left(\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{h} \right)}{\sum_{i=1}^{n} g\left(\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{h} \right)} \right] - \mathbf{x}$$



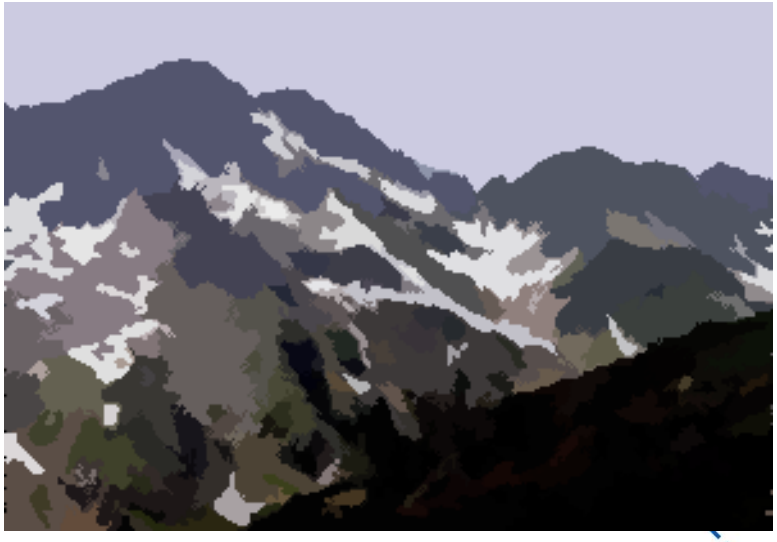
$$g(x_i - x) = e^{-c||x_i - x||^2}$$

Mean shift segmentation results



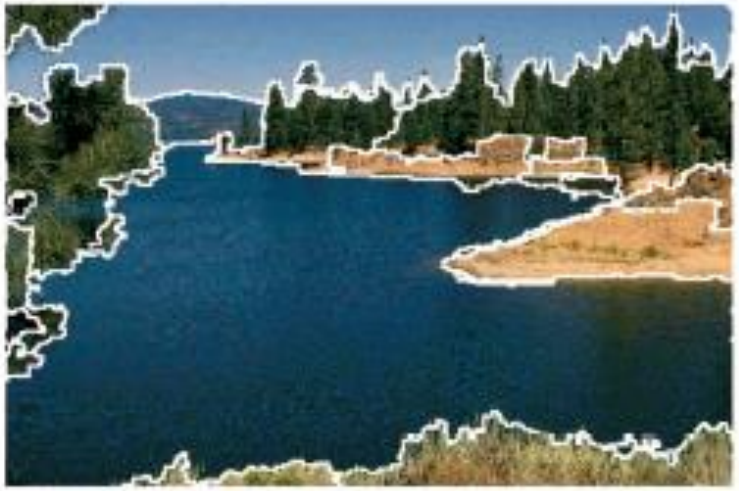






Mean shift segmentation results





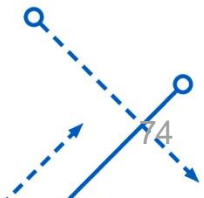




Comaniciu and Meer 2002

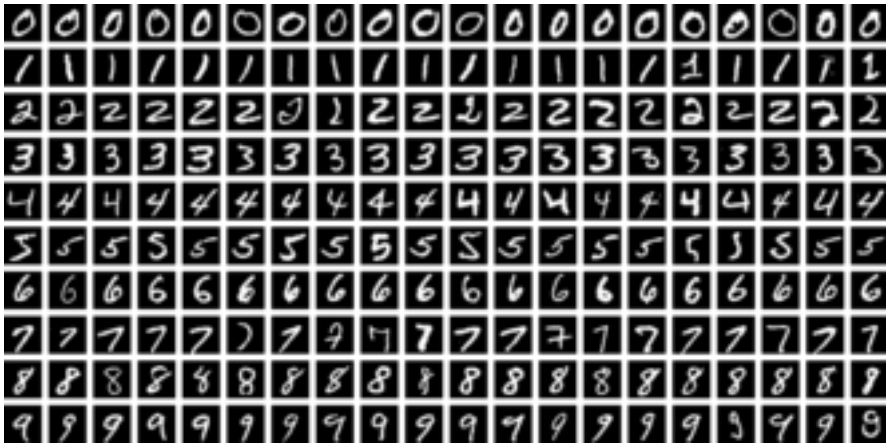
Mean shift pros and cons

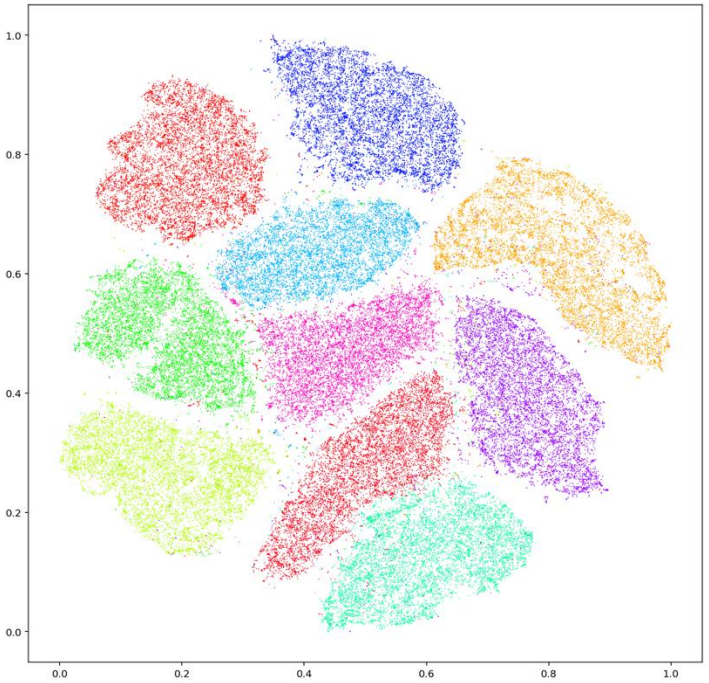
- Pros
  - Good general-practice segmentation
  - Flexible in number and shape of regions
  - Robust to outliers
- Cons
  - Have to choose kernel size in advance
    - Use kNN to determine window sizes adaptively
  - Not suitable for high-dimensional features
- When to use it
  - Oversegmentation, Multiple segmentations
  - Tracking, clustering, filtering applications



Application: t-SNE (Not in Syllabus)

- t-distributed stochastic neighbor embedding (t-SNE)
  - visualizing high-dimensional data by giving each datapoint a location in a 2- or 3-dimensional map.





T-SNE embeddings of MNIST dataset

Hinton, Geoffrey; Roweis, Sam (January 2002). Stochastic neighbor embedding (PDF). Neural Information Processing Systems.

t-SNE

- 1. Probabilities that x_i take x_j as its neighbor are proportional to the similarity of objects i and j.

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2 / 2\sigma_i^2)}$$

- 2. On the map, similarity of two points (Student t-distribution).

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}$$

- 3. Locations of y_i for objects x_i on the map is determined by minimizing the distance of two distributions (KL divergence).

$$\mathrm{KL}\left(P \parallel Q\right) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Hinton, Geoffrey; Roweis, Sam (January 2002). Stochastic neighbor embedding (PDF). Neural Information Processing Systems.

Things to remember

- Texture:
  - LBP and Uniform LBP algorithm
- Clustering
  - K-means
    - summarization, general clustering
  - Single link
    - segmentation, general clustering
  - Mean-shift
    - determining relevance, summarization, segmentation

